

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章：非线性系统的建模与仿真

Chapter 6 Modeling and Simulation of Nonlinear Systems



Professor Dingyu Xue, xuedingyu@mail.neu.edu.cn
School of Information Science and Engineering,
Northeastern University, Shenyang, CHINA



非线性系统的建模与仿真方法

- 前面各章涉及的是线性系统，非线性系统怎么办？
- 可以引入Simulink对系统建模、分析
- 本章主要内容
 - Simulink简介
 - 基于Simulink的框图绘制方法
 - 控制系统的建模仿真方法
 - 非线性系统的建模与仿真方法
 - Simulink的高级技术——子系统与S-函数
 - 多领域物理建模概述

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

Simulink简介

An Introduction to Simulink



主讲：薛定宇教授



Simulink建模基础

- 在Simulink出现之前，MATLAB仿真功能弱
 - 借助ACSL等仿真语言进行仿真
 - 接口不好，需要数据文件交互
 - 描述系统采用语言描述，缺乏框图支持，易错
- Simulink的出现与进展
 - 1990年MathWorks推出了SimuLAB，取代ACSL
 - 1992年更名Simulink
 - 2007年，支持Simscape、多领域物理建模
 - 2012年，2012b，全新的建模界面，使用方便，新版本功能更强



Simulink的常用模块简介

➤ 启动Simulink

➤ 命令行式

```
>> open_system('simulink')
```

➤ 双击图标——不同版本的Simulink图标略有不同

➤ 相关模块简介

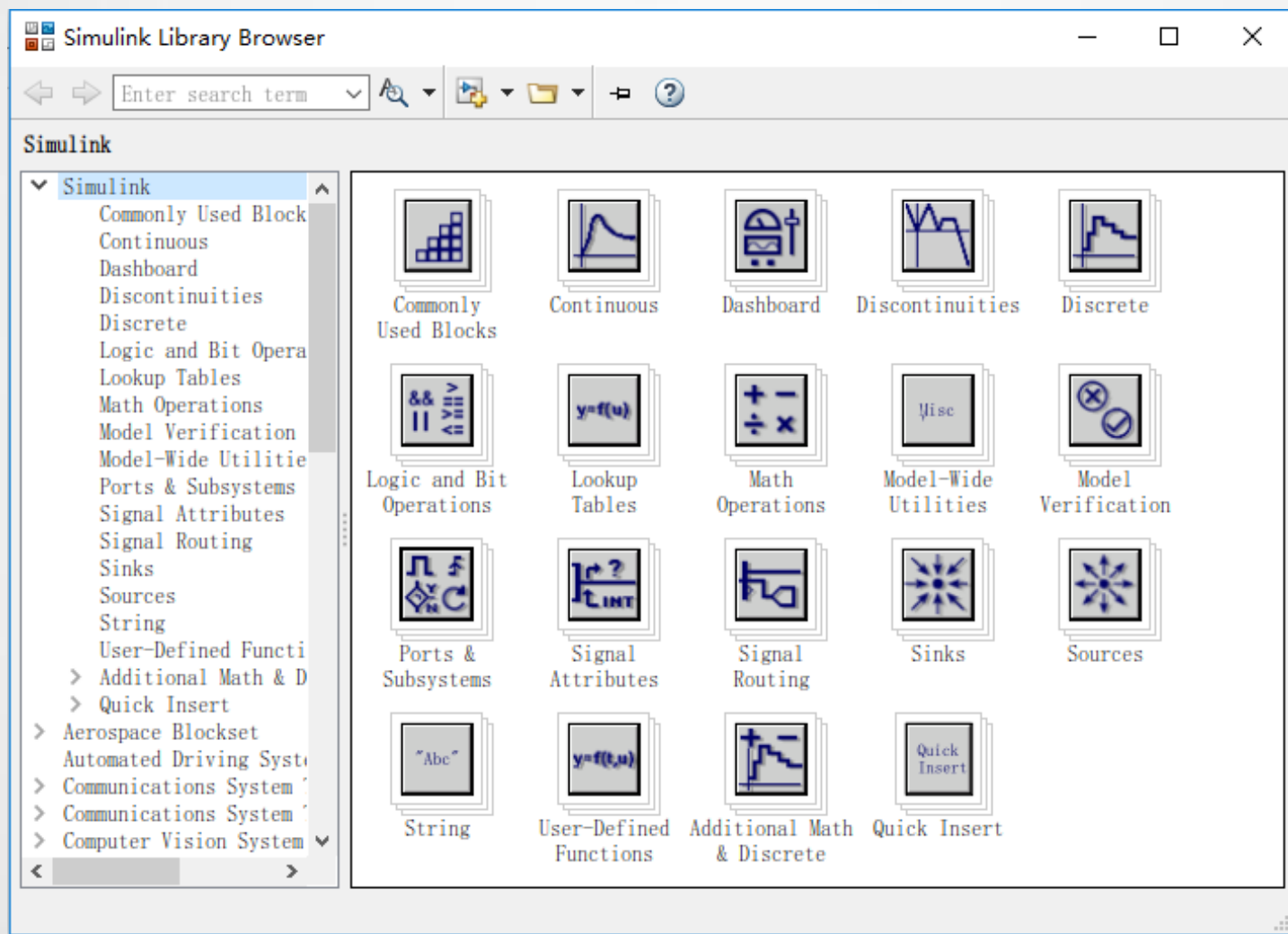
➤ 输入模块组、输出模块组

➤ 其他相关模块：连续、离散、非线性等

➤ 专业模块：Simscape、SimMechanics等



现有的模块库





输入模块组 Sources



Band-Limited
White Noise



Chirp Signal



Clock



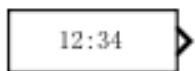
Constant



Counter
Free-Running



Counter
Limited



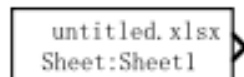
Digital Clock



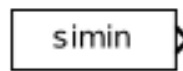
Enumerated
Constant



From File



From Spreadsheet



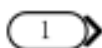
From
Workspace



Ground



In Bus Element



In1



Pulse
Generator



Ramp



Random
Number



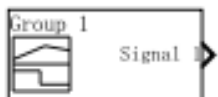
Repeating
Sequence



Repeating
Sequence
Interpolated



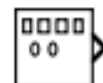
Repeating
Sequence
Stair



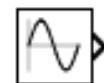
Signal Builder



Signal Editor



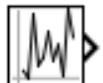
Signal
Generator



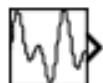
Sine Wave



Step



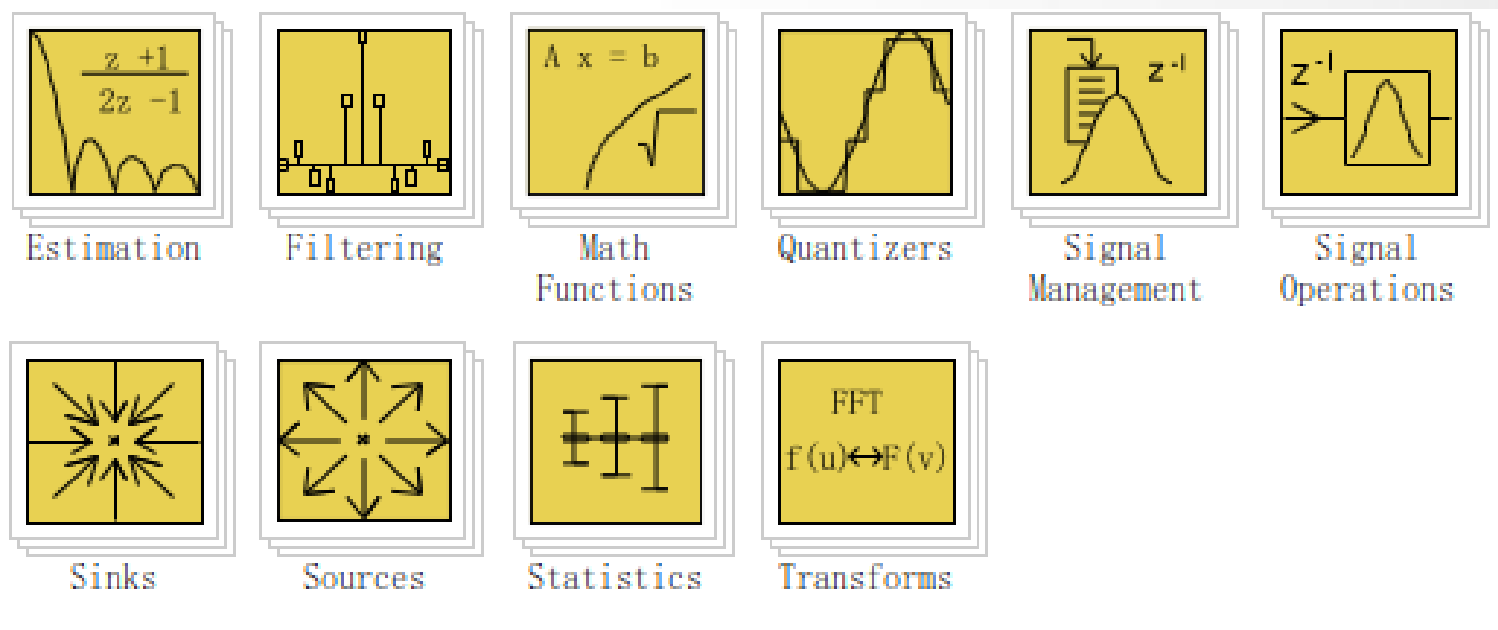
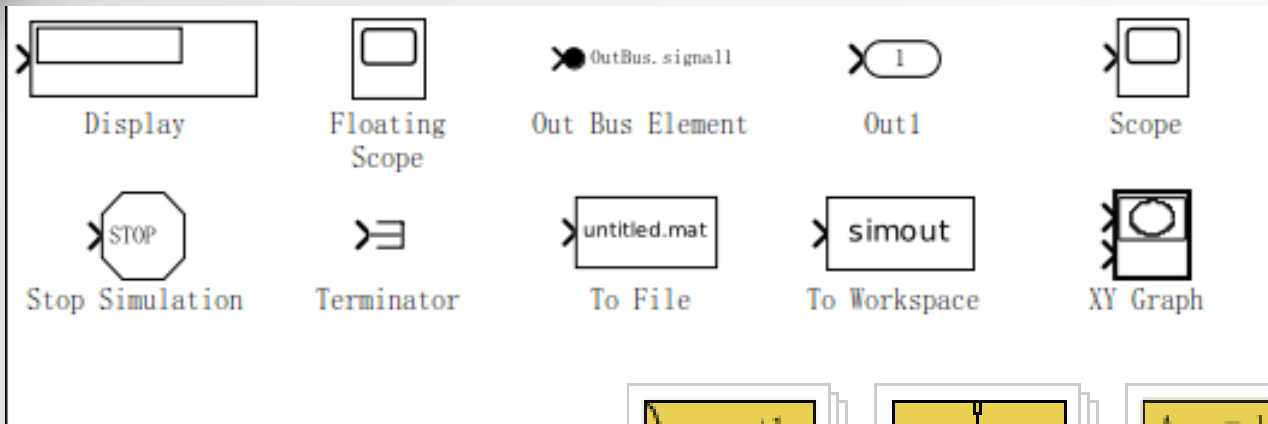
Uniform Random
Number



Waveform
Generator

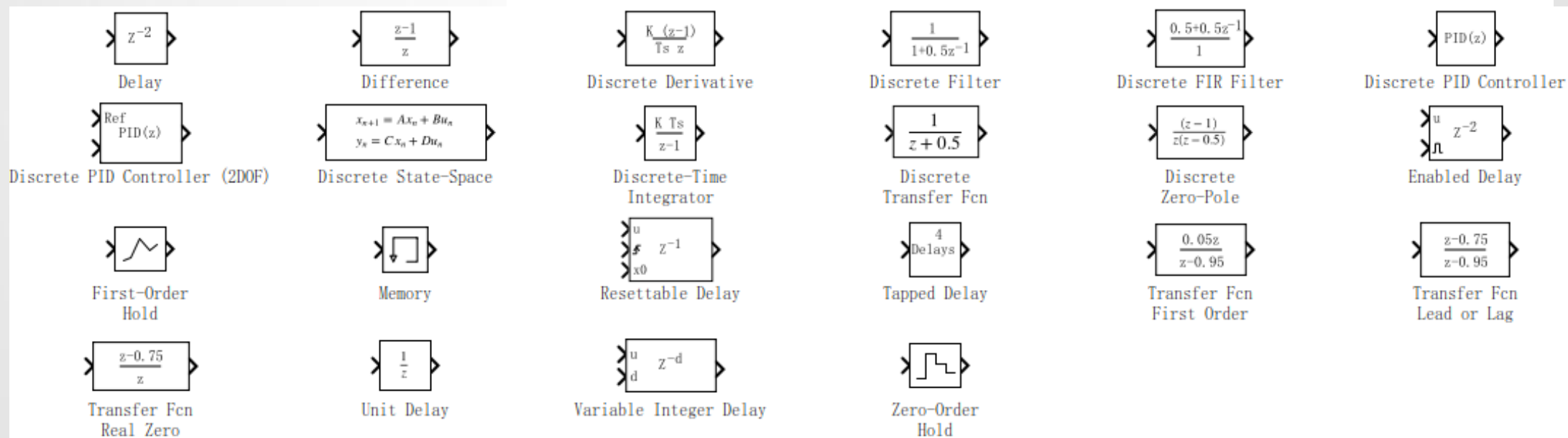
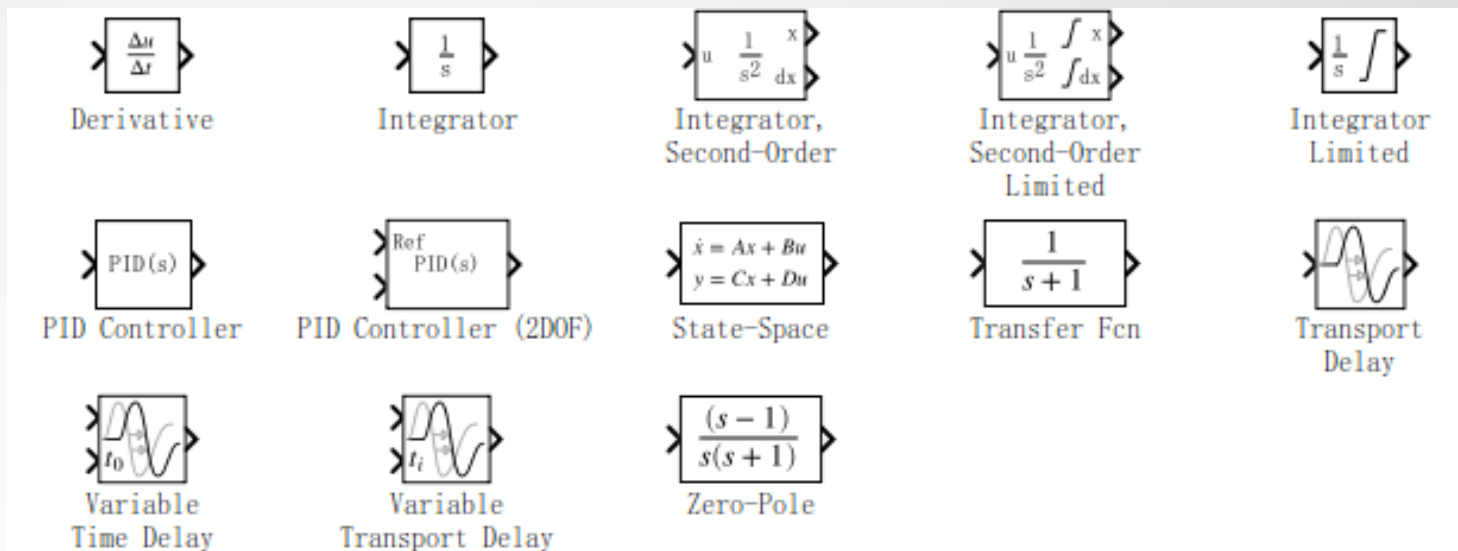


输出模块组



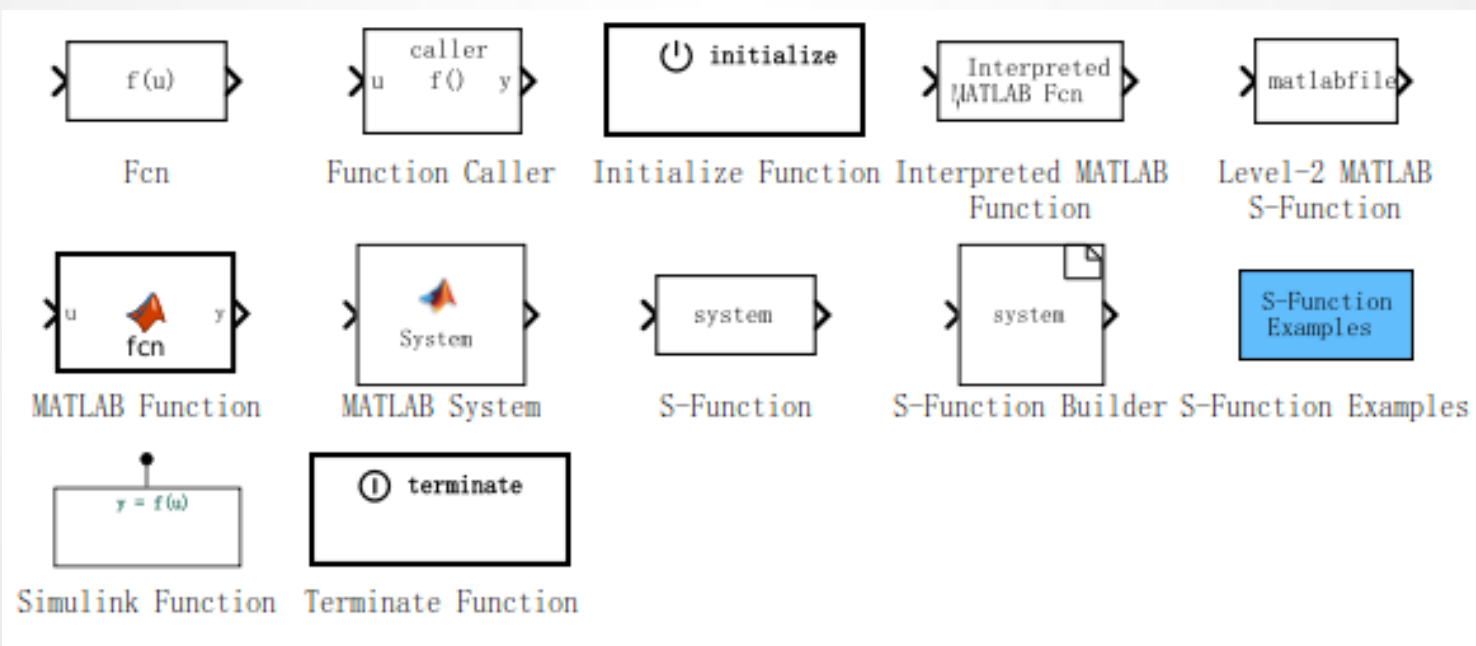
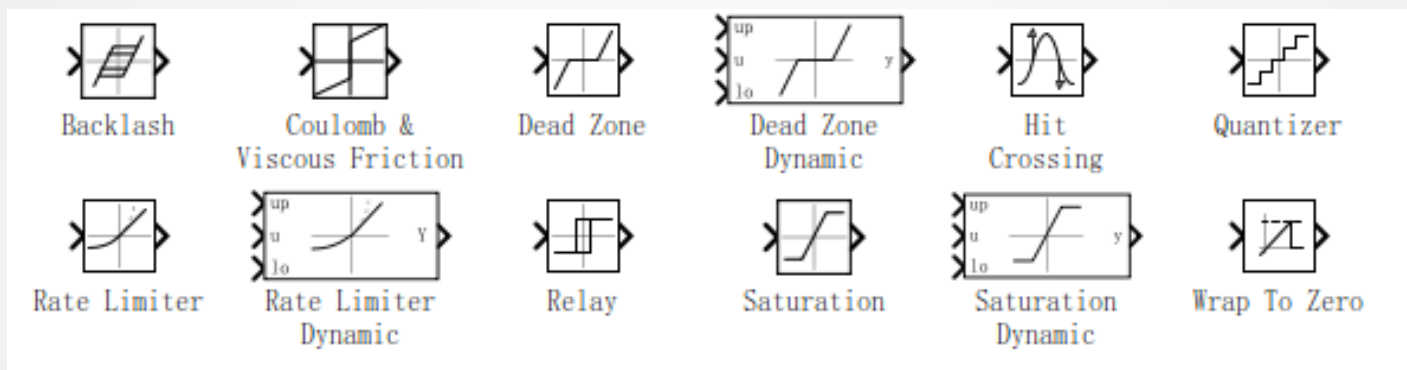


线性模块组





非线性模块与自定义模块组





Toolboxes and Blocksets

- > Aerospace Blockset
 - Automated Driving System Toolbox
 - > Communications System Toolbox
 - > Communications System Toolbox HDL Support
 - > Computer Vision System Toolbox
 - > Control System Toolbox
 - > Data Acquisition Toolbox
 - > DSP System Toolbox
 - > DSP System Toolbox HDL Support
 - > Embedded Coder
 - > Fuzzy Logic Toolbox
 - > HDL Coder
 - > HDL Verifier
 - > Image Acquisition Toolbox
 - > Instrument Control Toolbox
 - > MIMO FOTF Toolbox
 - > Model Predictive Control Toolbox
 - > Neural Network Toolbox
 - > OPC Toolbox
 - > Phased Array System Toolbox
 - > Powertrain Blockset
 - > Report Generator
 - > RF Blockset
 - > Robotics System Toolbox
 - > Robust Control Toolbox
 - > SimEvents
- > Simscape
 - > Simulink 3D Animation
 - > Simulink Coder
 - > Simulink Control Design
 - > Simulink Design Optimization
 - > Simulink Design Verifier
 - > Simulink Desktop Real-Time
 - > Simulink Extras
 - > Simulink Real-Time
 - > Simulink Requirements
 - > Simulink Test
 - > Stateflow
 - > System Identification Toolbox
 - > Vehicle Network Toolbox
 - > Vision HDL Toolbox
 - > Recently Used



Foundation
Library



Utilities



Driveline



Electronics



Fluids



Multibody



Power Systems

Simscape 4.4
Copyright 2006-2018 The MathWorks, Inc.



Simulink小结

- 简单介绍了Simulink发展背景
- 介绍了一些常用的模块
 - 输入模块
 - 输出模块
 - 线性连续模块、线性离散模块
 - 非线性模块



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

Simulink 建模与仿真举例

An Illustrative Example of Simulink Modeling and Simulation



主讲：薛定宇教授



Simulink的建模与仿真的步骤

➤ 建模步骤

- 打开模块库
- 打开空白窗口
- 将所需模块复制到模型窗口内
- 修改参数、连线
- 使用技巧——信号显示、模块连接、自动翻转

➤ 仿真步骤

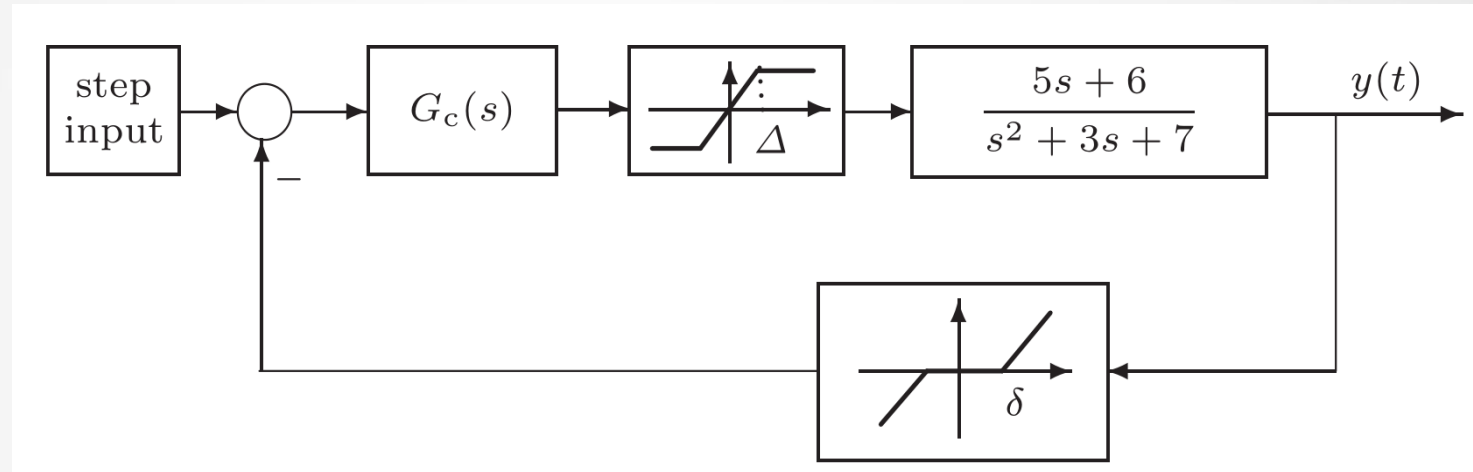
- 仿真参数
- 启动仿真——修改参数再仿真



例6-1 非线性系统建模与仿真

➤ 非线性系统建模与仿真

$$G_c(s) = 3 + 2/s$$
$$\Delta = 2, \delta = 0.1$$



➤ 所需模块：

- 传递函数、饱和、死区、加法器、输入、输出
- 模块修饰、模块连线、仿真参数、启动仿真
- 修改参数



Simulink例子小结

- Simulink建模的步骤
 - 打开空白模型窗口
 - 找出所需模块并复制到窗口
 - 修改参数
 - 连线，构造整个模型，使用技巧
- 模型的仿真
 - 控制参数修改
 - 仿真结果的检验——不同的算法、相对误差限



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

微分方程的Simulink建模与求解

Simulink Modeling and Solutions of Ordinary Differential Equations



主讲：薛定宇教授



微分方程建模基础

- 微分方程是动态连续系统的重要表现形式
- 本节给出一种通用的描述方法



6.2 仿真举例——微分方程建模

➤ 微分方程的建模：Rössler方程

$$\begin{cases} x'(t) = -y(t) - z(t) \\ y'(t) = x(t) + ay(t) \\ z'(t) = b + [x(t) - c]z(t) \end{cases}$$

$$\begin{aligned} a &= b = 0.2, c = 5.7 \\ x(0) &= y(0) = z(0) = 0 \end{aligned}$$

➤ 建模方法

- 底层建模——关键信号定义：积分器使用
- 利用MATLAB函数模块 Fcn



微分方程最底层的建模方法

➤ 底层模块建模

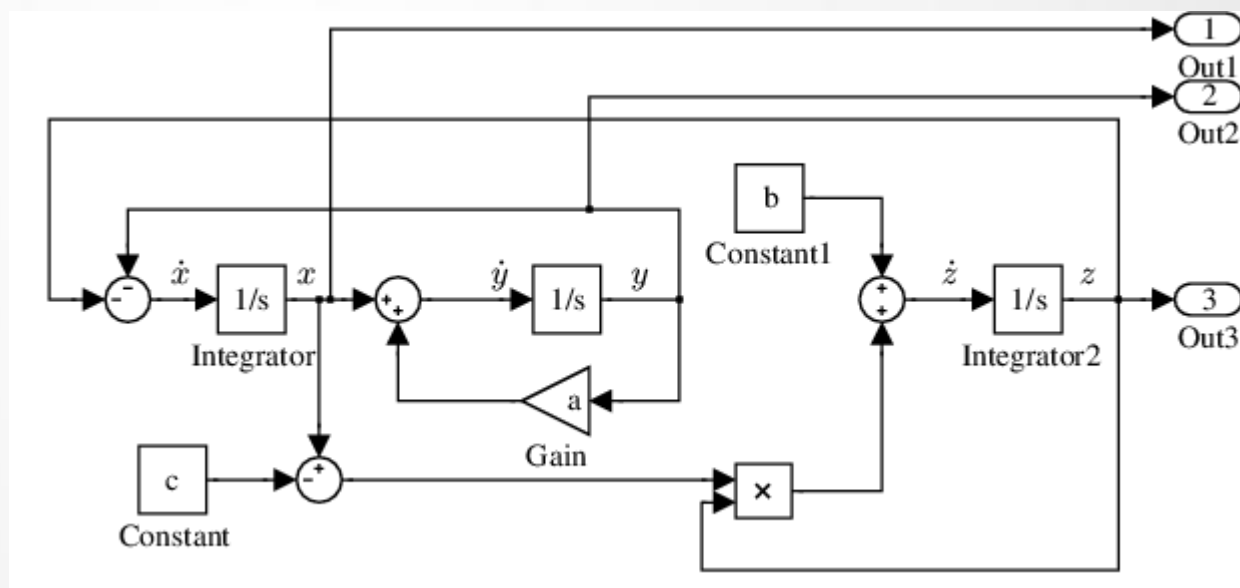
➤ 先构建关键信号

x, x', y, y', z, z'

➤ 由关键信号连线， 画出微分方程

➤ 微分方程的求解： 仿真方法

$$\begin{cases} x'(t) = -y(t) - z(t) \\ y'(t) = x(t) + ay(t) \\ z'(t) = b + [x(t) - c]z(t) \end{cases}$$





简单建模方法

➤ 简化模型

➤ 为什么需要简化？

➤ 利用Fcn模块——局限性

➤ 模型修饰——向量信号、数据类型、路数等

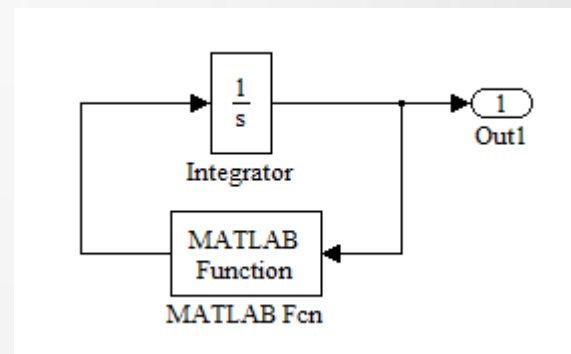
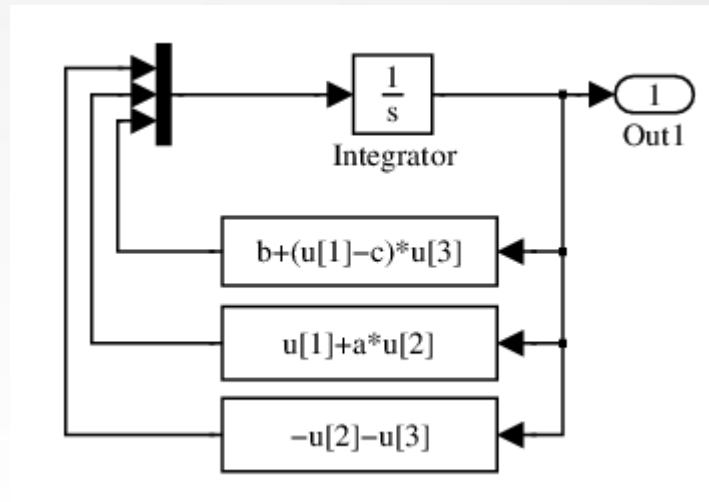
➤ 更简单的描述：

➤ MATLAB Fcn模块——模型：c6mross1.mdl

```
function y=c6mross(x)
```

```
a=0.2; b=0.2; c=5.7;
```

```
y=[-x(2)-x(3); x(1)+a*x(2); b+[x(1)-c]*x(3)];
```





微分方程的求解

➤ MATLAB求解

```
>> a=0.2; b=0.2; c=5.7; x0=[0; 0; 0];  
    f=@(t,x)[-x(2)-x(3); x(1)+a*x(2); b+[x(1)-c]*x(3)];  
    [t,x]=ode45(f,[0,10],x0); plot(t,x)
```

➤ Simulink建模优势：模块化建模

➤ 两种方法

➤ 界面方法：输出端子——返回 tout, yout

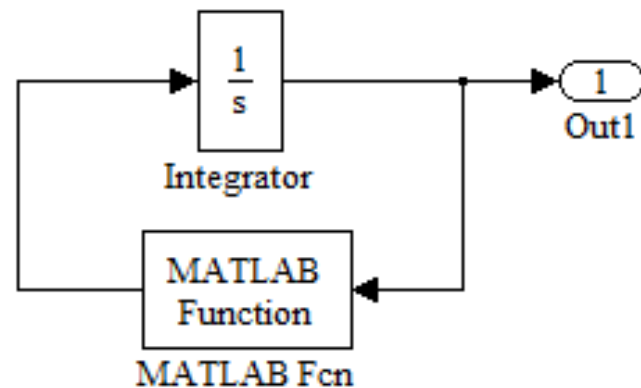
➤ 语句方法

```
[t,x,y]=sim(model,[t0,tf],options)
```



微分方程Simulink建模小结

- 用Simulink画出微分方程
 - 关键信号构造——利用积分器搭建
 - 向量化的积分器
 - 自定义函数描述微分方程
 - 统一的描述框架
- 微分方程求解——sim函数





国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

控制系统的Simulink 建模举例 (上)

Simulink Modeling Examples of Control Systems (I)



主讲：薛定宇教授



控制系统的建模与仿真举例

- Simulink 提供的工具可以把控制系统画出来
- 这里将介绍各种控制系统的建模仿真方法
 - 多变量控制系统
 - 计算机控制系统
 - 时变系统
 - 多采样速率系统
 - 延迟系统与变延迟系统
 - 切换系统
 - 随机输入系统



控制系统可用的模块

➤ 线性系统模块

➤ Simulink自带的Continuous模块组、Discrete模块组

➤ 标准的传递函数、状态方程、零极点、延迟、积分器、PID控制器

➤ Simulink Extras —— 非零初值的传递函数等

➤ 控制系统工具箱模块集——LTI对象模块

➤ 非线性系统模块

➤ 下节介绍



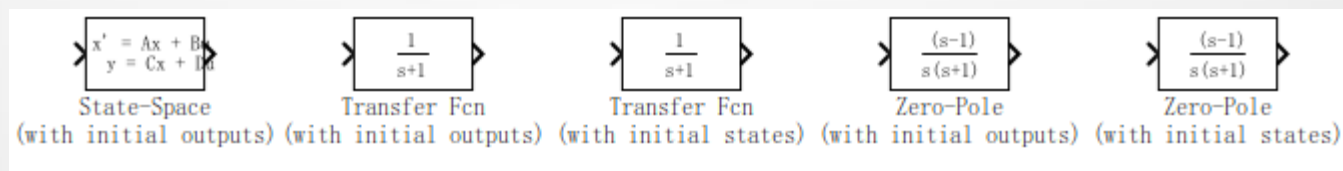
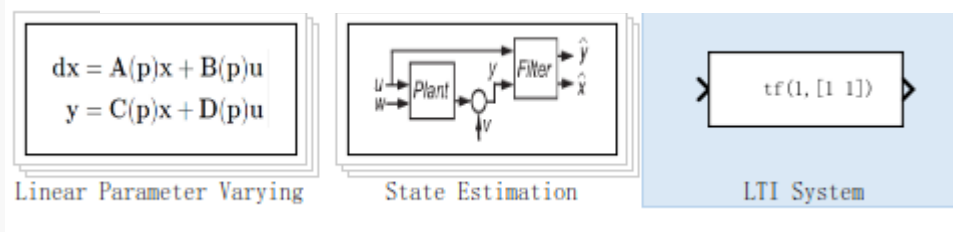
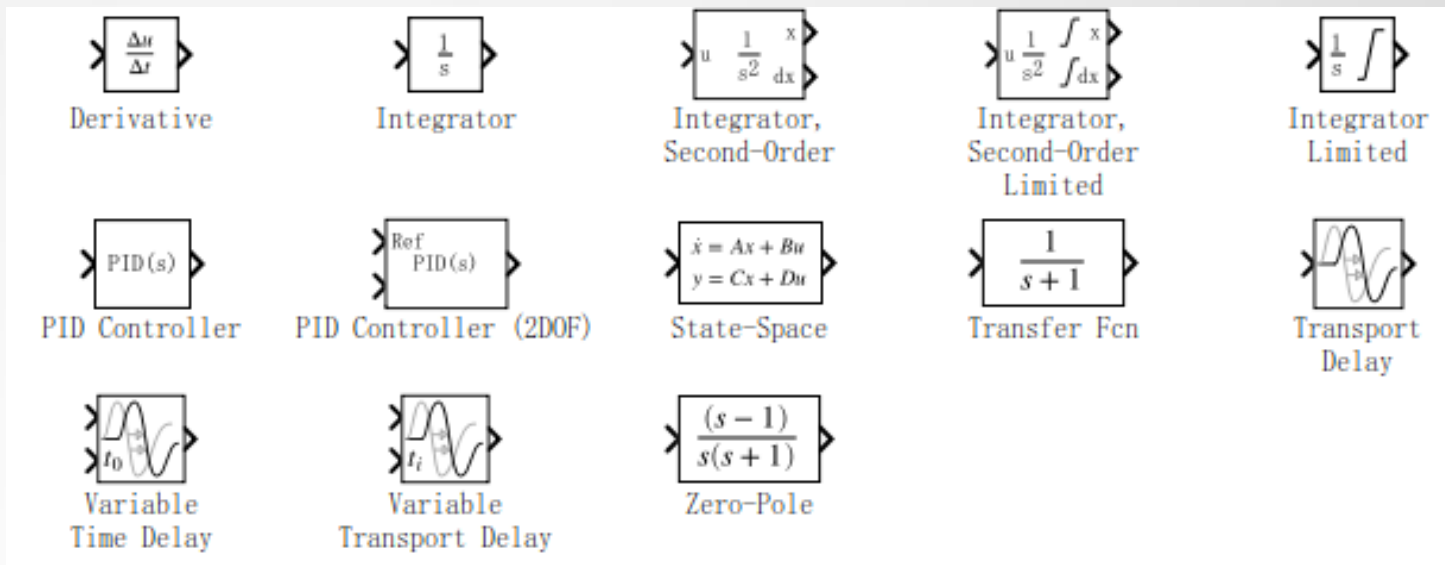
线性模块库

➤ 常规模块库

➤ 控制系统工具箱

➤ 非零初值系统的建模

➤ Simulink Extras -> Additional Linear





例6-3多变量时间延迟系统的仿真

➤ 多变量系统

$$G(s) = \begin{bmatrix} \frac{0.1134e^{-0.72s}}{1.78s^2 + 4.48s + 1} & \frac{0.924}{2.07s + 1} \\ \frac{0.3378e^{-0.3s}}{0.361s^2 + 1.09s + 1} & \frac{-0.318e^{-1.29s}}{2.93s + 1} \end{bmatrix}$$

➤ MATLAB仿真方法——局限性

➤ Simulink仿真



MATLAB求解方法

➤ 补偿系统的阶跃响应

```
 >> g11=tf(0.1134,[1.78 4.48 1],'ioDelay',0.72);  
      g21=tf(0.3378,[0.361 1.09 1],'ioDelay',0.3);  
      g12=tf(0.924,[2.07 1]);  
      g22=tf(-0.318,[2.93 1],'ioDelay',1.29);  
      G=[g11, g12; g21, g22]; G=ss(G);  
      Kp=[0.1134,0.924; 0.3378,-0.318]; step(G*Kp,15)
```

➤ 局限性——如果有非线性环节，无能为力

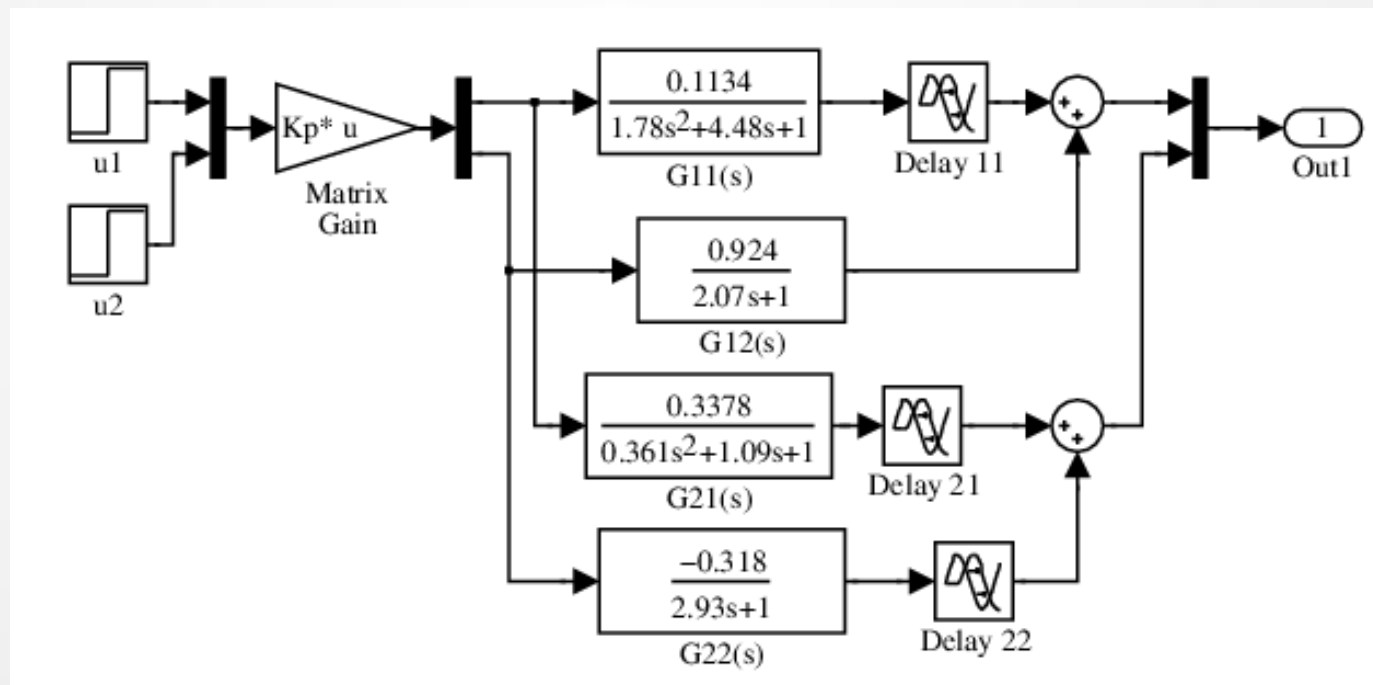


Simulink仿真方法

➤ 早期版本建模方法

➤ 模型文件：c6mmimo.mdl

$$G(s) = \begin{bmatrix} \frac{0.1134e^{-0.72s}}{1.78s^2 + 4.48s + 1} & \frac{0.924}{2.07s + 1} \\ \frac{0.3378e^{-0.3s}}{0.361s^2 + 1.09s + 1} & \frac{-0.318e^{-1.29s}}{2.93s + 1} \end{bmatrix}$$





使用**LTI**模块的建模仿真方法

➤ 直接建模—— c6mmimon.slx

➤ 不同方法比较



```
>> u1=1; u2=0; [t1,a,y1]=sim('c6mmimo',15);  
    u1=0; u2=1; [t2,a,y2]=sim('c6mmimo',15);  
    [y,t]=step(G*Kp,15);
```



```
>> subplot(221), plot(t,y(:,1,1),':',t1,y1(:,1))  
    subplot(222), plot(t,y(:,1,2),':',t2,y2(:,1))  
    subplot(223), plot(t,y(:,2,1),':',t1,y1(:,2))  
    subplot(224), plot(t,y(:,2,2),':',t2,y2(:,2))
```

➤ 对复杂线性系统模型，尽量采用新版的LTI模块



例6-4 复杂系统模型的建模

- 复杂系统模型，如何建模？

$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$$

- 底层建模

$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1} \rightarrow \frac{1}{s+1} \left(1 + \frac{3e^{-s}}{s+1} \right)$$

- 如果采用LTI，带有内部延迟的状态方程模型



```
>> s=tf('s'); G1=3/(s+1); G1.ioDelay=3; G=(1+G1)/(s+1)
```



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

控制系统的Simulink建模举例 (中)

Simulink Modeling Examples of Control Systems (II)

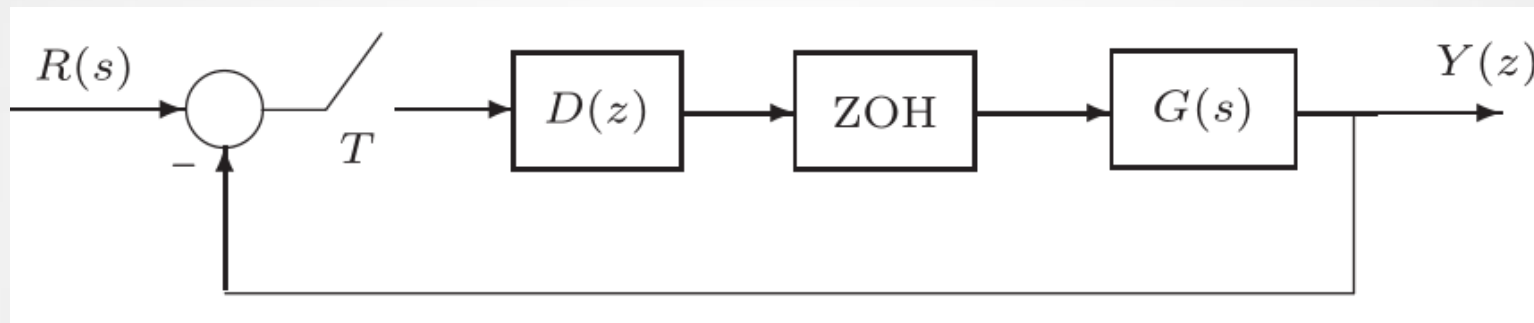


主讲：薛定宇教授



例6-5 计算机控制系统仿真

➤ 计算机控制系统



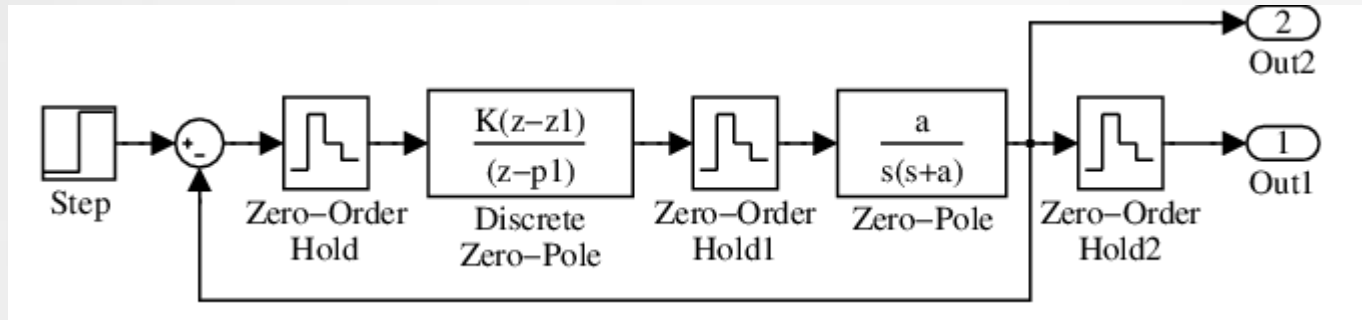
$$G(s) = \frac{a}{s(s+1)}, \quad D(z) = \frac{1 - e^{-T}}{1 - e^{-0.1T}} \frac{z - e^{-0.1T}}{z - e^{-T}}$$

➤ 所需模块

- 连续传递函数、离散传递函数、零阶保持器
- 模型名：c6mcompc.mdl



Simulink模型



```
>> T=0.2; a=0.1; z1=exp(-0.1*T); p1=exp(-T);  
K=(1-p1)/(1-z1); [t,x,y]=sim('c6mcompc',20);  
plot(t,y(:,2)); hold on; stairs(t,y(:,1))
```



```
>> T=1; z1=exp(-0.1*T); p1=exp(-T); K=(1-p1)/(1-z1);  
[t,x,y]=sim('c6mcompc',20); plot(t,y(:,2));  
hold on; stairs(t,y(:,1))
```



```
>> T=0.2; z1=exp(-0.1*T); p1=exp(-T); K=(1-p1)/(1-z1);  
Dz=zpk(z1,p1,K,'Ts',T); G=zpk([], [0;-a], a);  
Gz=c2d(G,T); GG=zpk(feedback(Gz*Dz,1)), step(GG)
```



Simulink模型的化简

➤ Simulink模型进一步化简

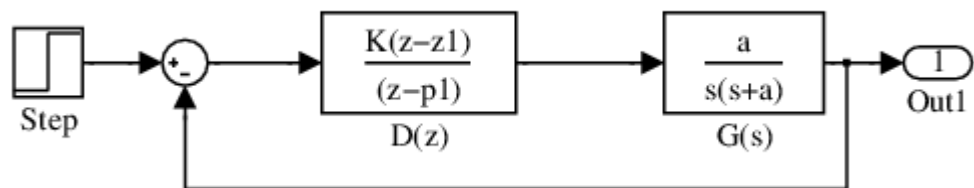
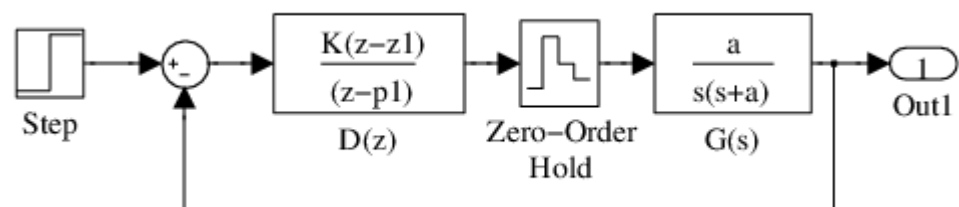
➤ c6mcomc1

➤ c6mcomc2

➤ 参数的预置

➤ File -> Model properties

$T=0.2$; $z1=\exp(-0.1*T)$; $p1=\exp(-T)$; $K=(1-p1)/(1-z1)$;



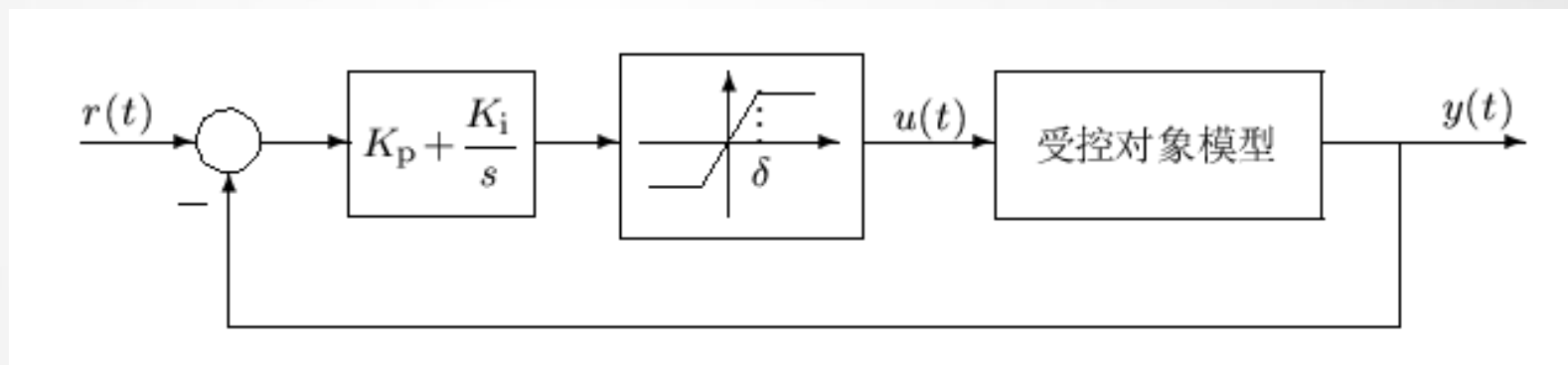


例6-6 时变系统仿真

➤ 受控对象为微分方程

$$\ddot{y}(t) + e^{-0.2t}\dot{y}(t) + e^{-5t}\sin(2t + 6)y(t) = u(t)$$

➤ PI 控制器



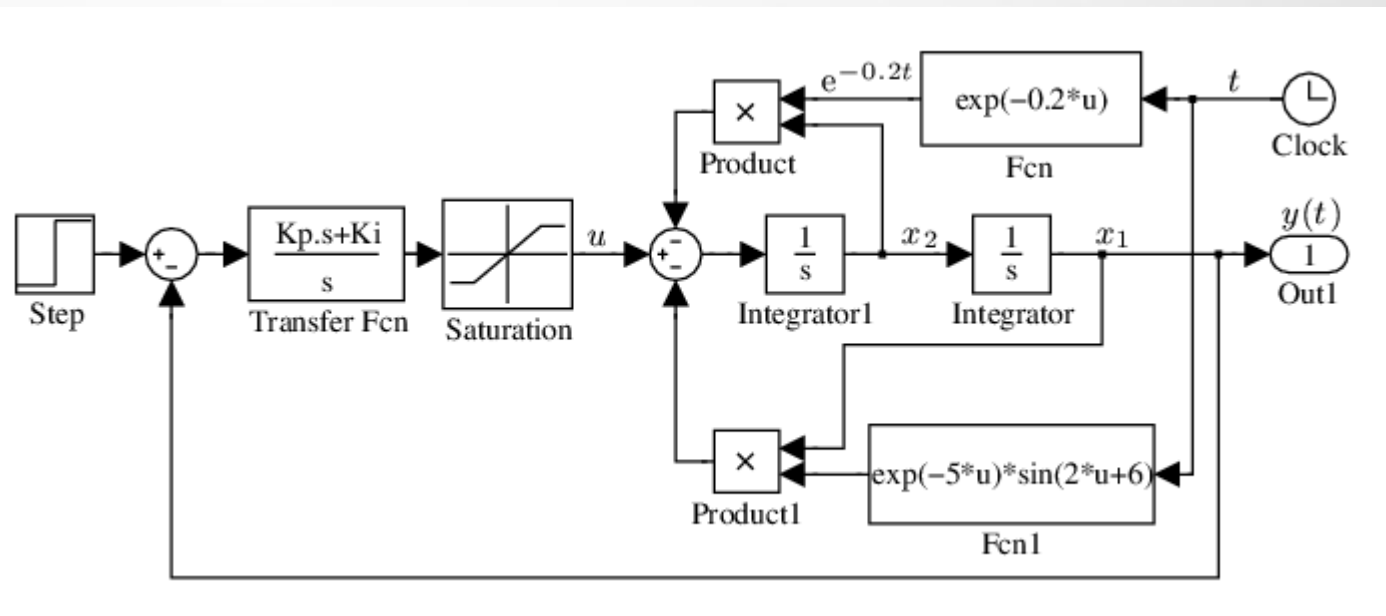
➤ 改写受控对象模型

$$\ddot{y}(t) = -e^{-0.2t}\dot{y}(t) - e^{-5t}\sin(2t + 6)y(t) + u(t)$$



Simulink建模

- 时变受控对象 $\ddot{y}(t) = -e^{-0.2t}\dot{y}(t) - e^{-5t}\sin(2t+6)y(t) + u(t)$
- 模型：c6mtimv.mdl

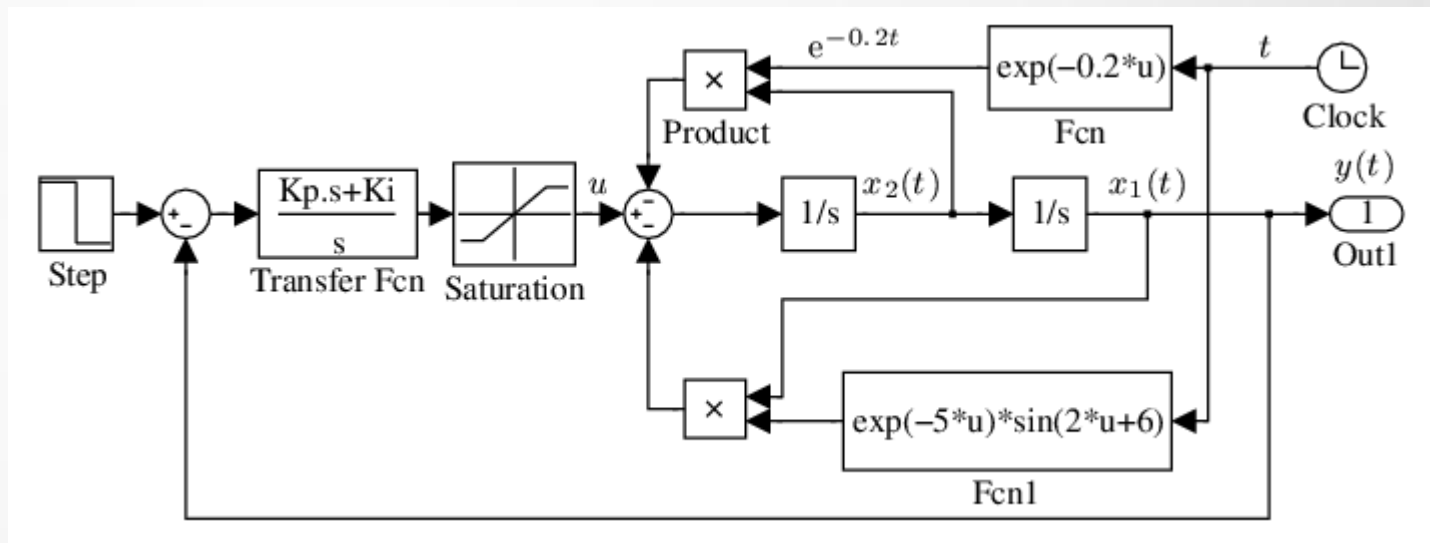
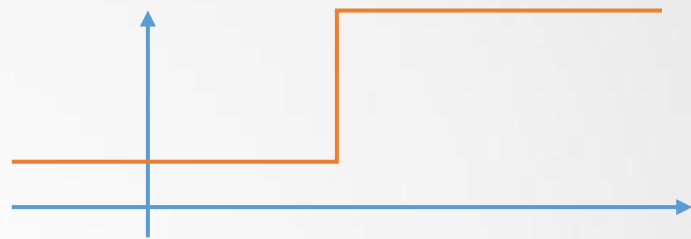


```
>> opt=simset('RelTol',1e-8); Kp=200; Ki=10;  
[t,x,y]=sim('c6mtimv',10,opt); plot(t,y)
```




例6-7 时变系统的脉冲响应

- 脉冲响应怎么求？
- 没有脉冲模块
- 脉冲模块的近似

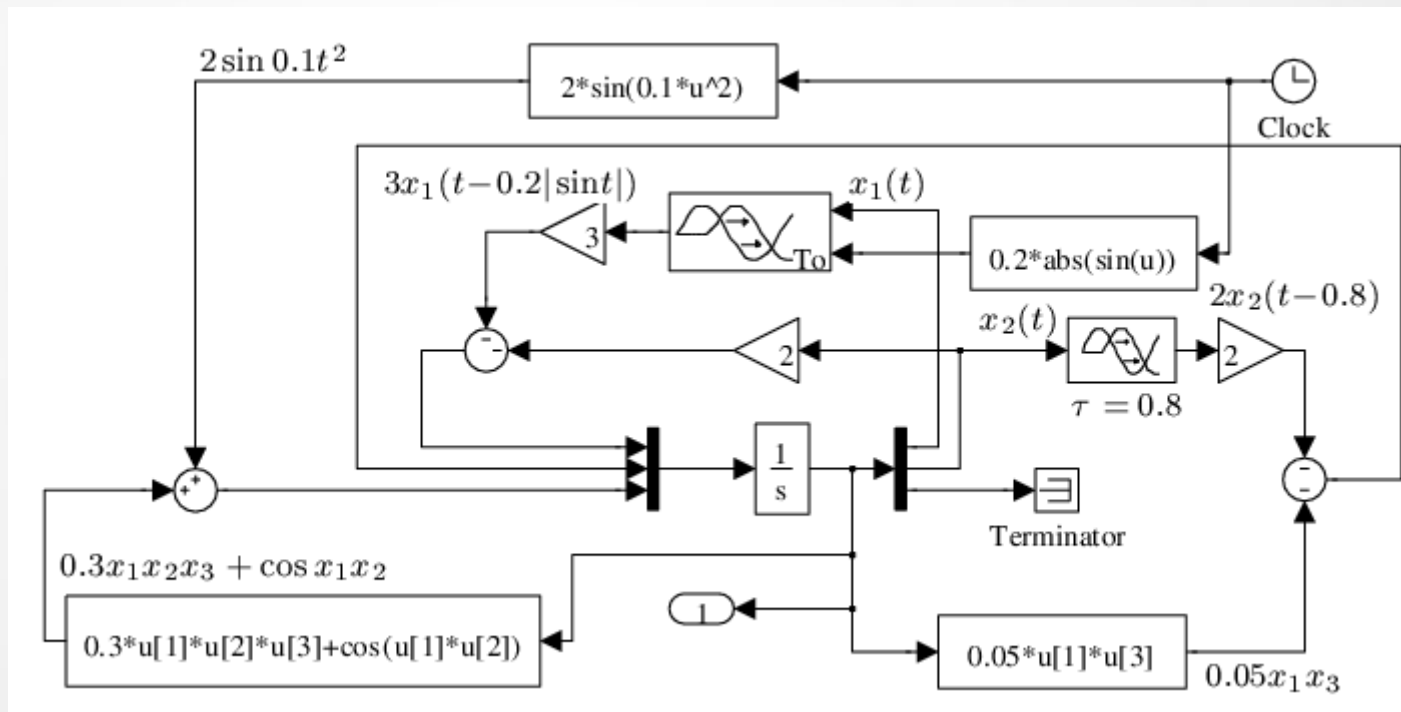
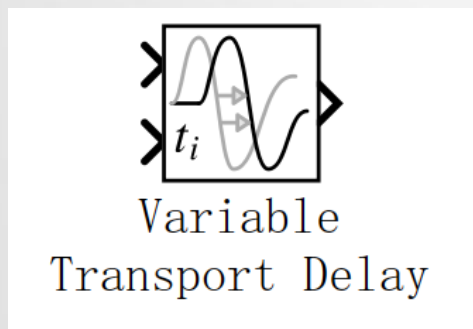
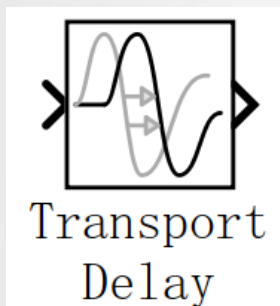


```
>> opt=simset('RelTol',1e-8); Kp=200; Ki=10; a=0.001;  
[t,x,y]=sim('c6mtimva',10,opt); plot(t,y)
```



变延迟

$$\begin{cases} \dot{x}_1(t) = -2x_2(t) - 3x_1(t - 0.2|\sin t|) \\ \dot{x}_2(t) = -0.05x_1(t)x_3(t) - 2x_2(t - 0.8) \\ \dot{x}_3(t) = 0.3x_1(t)x_2(t)x_3(t) + \cos(x_1(t)x_2(t)) + 2\sin 0.1t^2 \end{cases}$$





国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

控制系统的Simulink建模举例 (下)

Simulink Modeling Examples of Control Systems (III)



主讲：薛定宇教授



例6-9 切换系统的仿真

- 状态方程（自治系统） $\dot{x} = A_i x, x_1(0) = x_2(0) = 5$

$$A_1 = \begin{bmatrix} 0.1 & -1 \\ 2 & 0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0.1 & -2 \\ 1 & 0.1 \end{bmatrix}$$

- 不稳定矩阵，切换条件

- II、IV象限： $x_1 x_2 < 0 \rightarrow A_1$

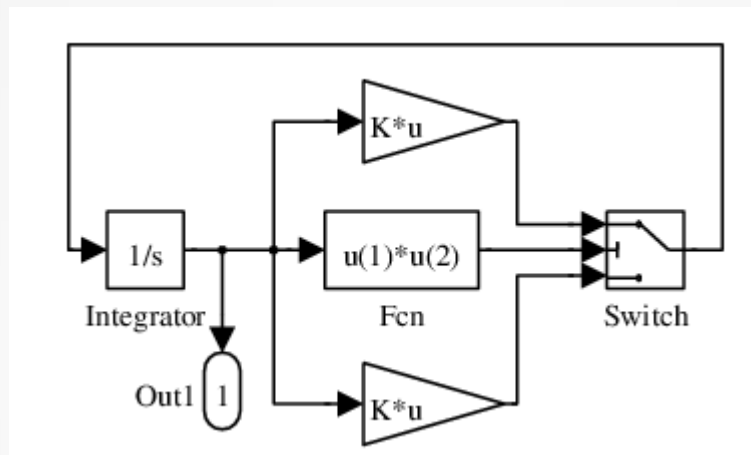
- I、III象限： $x_1 x_2 \geq 0 \rightarrow A_2$

- Simulink模型：c6mswi1.mdl——矩阵自动调入



Simulink建模方法

➤ 过零检测



➤ 相平面图



```
>> plot(tout,yout),
```



```
>> plot(yout(:,1),yout(:,2))
```

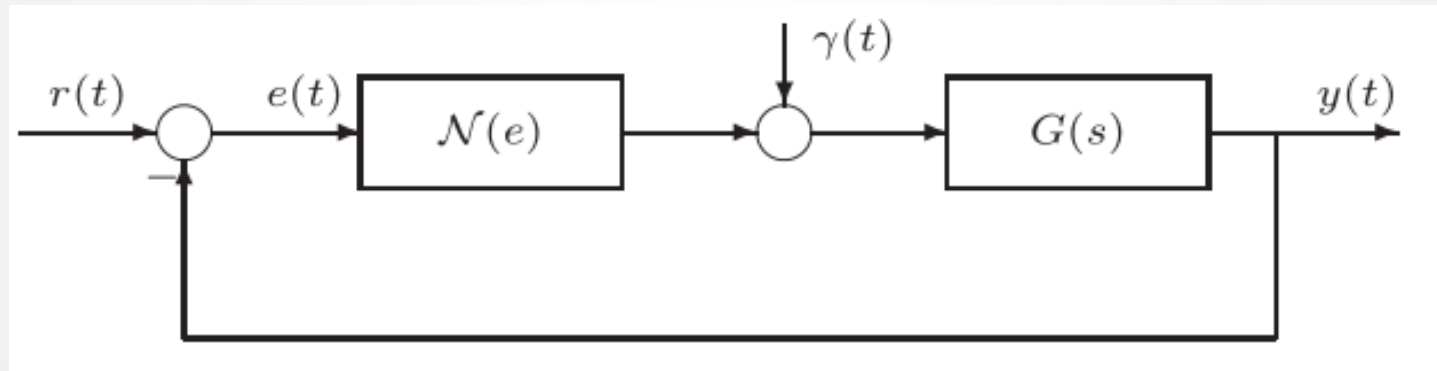
➤ 切换系统稳定



例6-10 随机输入系统的建模与仿真

➤ 非线性系统

$$G(s) = \frac{s^3 + 7s^2 + 24s + 24}{s^4 + 10s^3 + 35s^2 + 50s + 24}, \mathcal{N}(e) = \begin{cases} 2\text{sign}(e), & |e| > 1 \\ 2e, & |e| \leq 1 \end{cases}$$



➤ 所需模块

➤ 白噪声输入：Band-limited White Noise



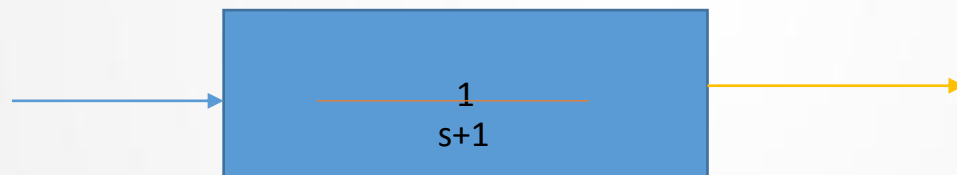
随机系统仿真的复杂性

➤ 以往很多文献中仿真方法都是错的！

➤ 随机数模块去激励连续模型存在问题，强度不足

$$u_1 = \frac{u}{\sqrt{\Delta T}}$$

➤ 检验



➤ 必须采用随机过程模拟信号激励

➤ Ornstein-Uhlenbeck随机过程

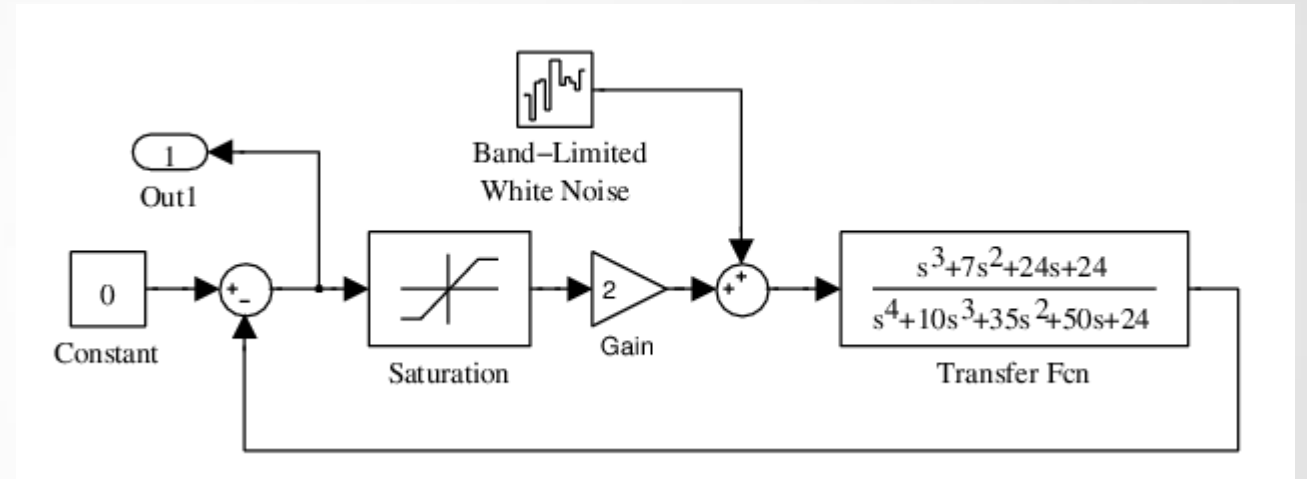
➤ Band Ltd White Noise



Simulink 仿真模型

➤ 仿真模型

➤ 模型名：c6mnlrsys.mdl



➤ 概率密度绘制



```
>> plot(tout(end-500:end),yout(end-500:end))  
      c=linspace(-2,2,20); y1=hist(yout,c);
```



```
>> bar(c,y1/(length(tout)*(c(2)-c(1))))
```



控制系统建模仿真小节

- 介绍了不同系统结构的Simulink建模方法
- 对线性系统模块建议采用LTI模块
- 尤其指出了随机输入信号传统仿真方法的错误



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

静态非线性环节

Static Nonlinear Elements Modeling



主讲：薛定宇教授



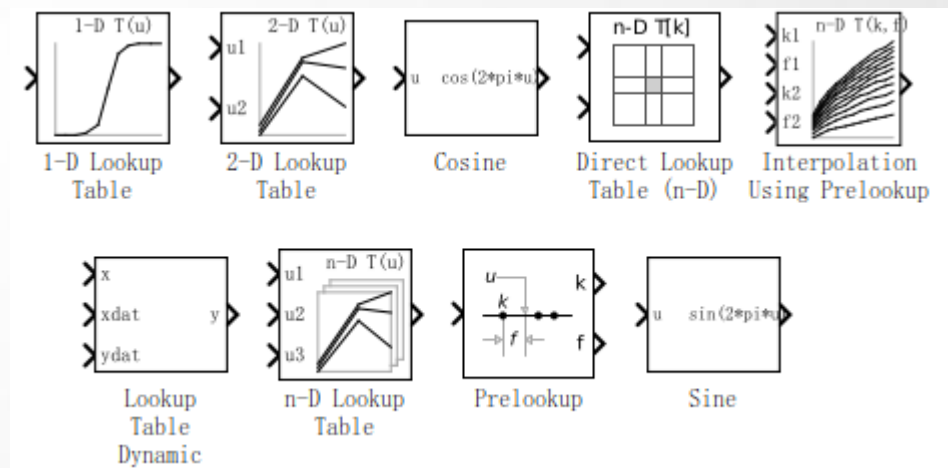
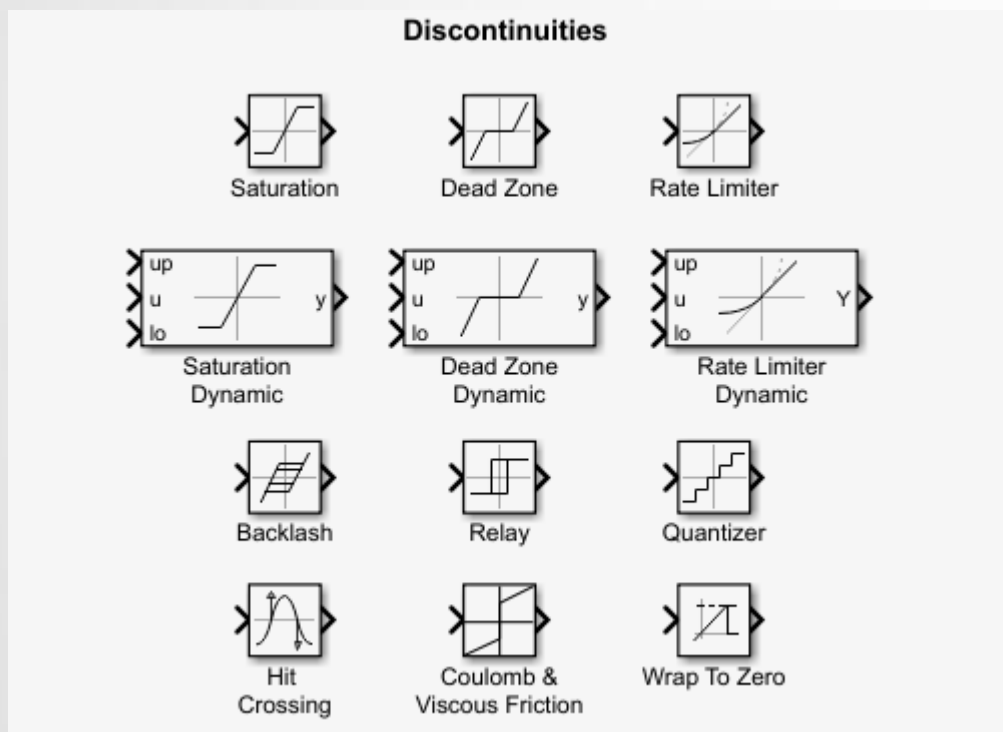
一般非线性系统仿真

- MATLAB控制系统工具箱直接用于线性时不变系统的分析与仿真
- 时变系统需要Simulink建模
- 现有非线性模块集的不足
- 一般静态非线性环节的建模
 - 单值非线性与多值非线性
 - 记忆非线性环节



现有非线性模块库

➤ Discontinuity模块组——模块有限难以描述任意非线性





一般静态非线性特性的建模

➤ 静态非线性： $y = f(u)$

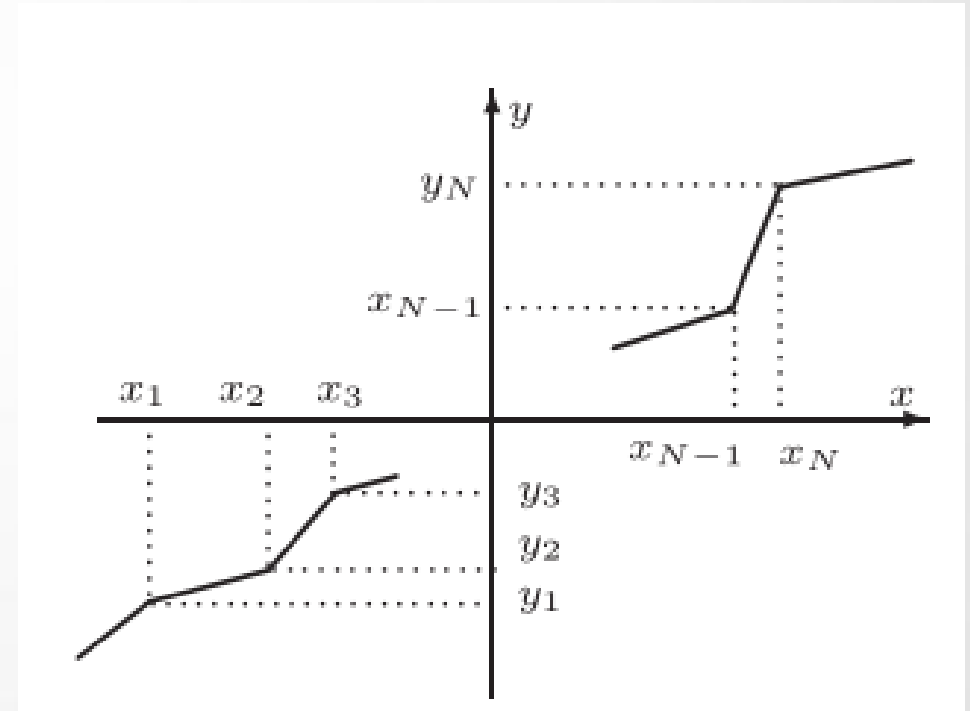
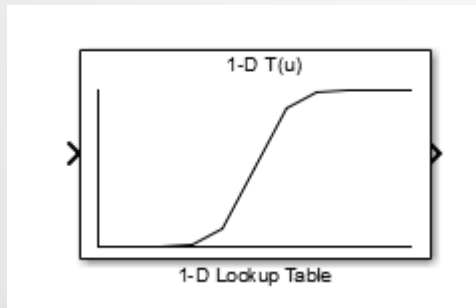
➤ 单值非线性环节

➤ 转折点

$xx = [x_0, x_1, x_2, \dots, x_N, x_{N+1}]$;

$yy = [y_0, y_1, y_2, \dots, y_N, y_{N+1}]$;

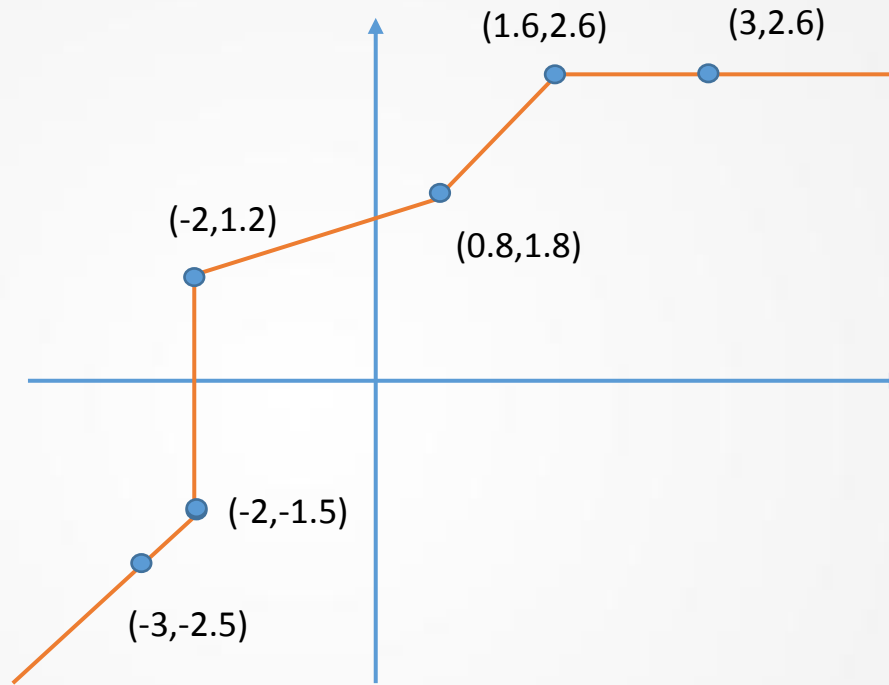
➤ 一维查表模块





例6-10 单值非线性环节的建模

- 单值非线性
 - 找出转折点



- 利用查表包括实现
- 如何处理直上直下的那条线

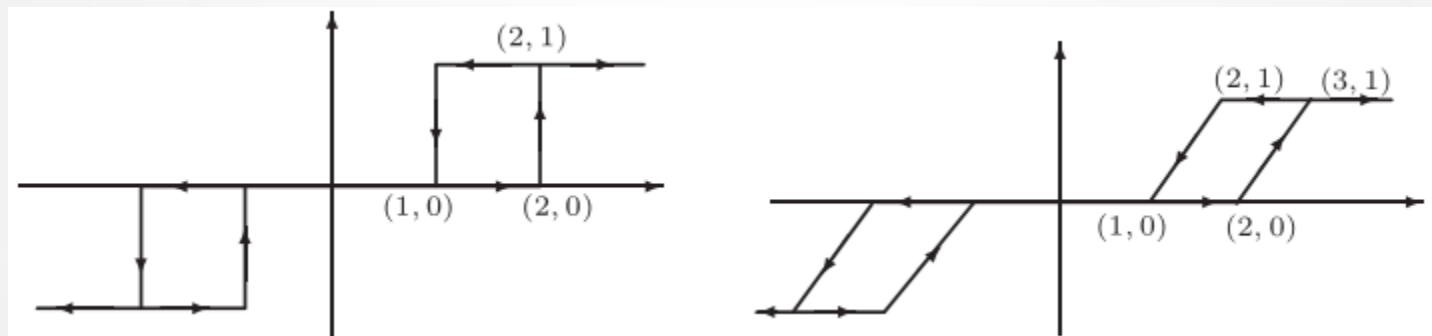


```
>> xx=[-3,-2,-2+eps,0.8,1.6,3]; yy=[-2.5 -1.5 1.2 1.8 2.6 2.6];
```



多值非线性环节建模

- 多值非线性在实际系统中存在——磁滞
- 双值非线性举例

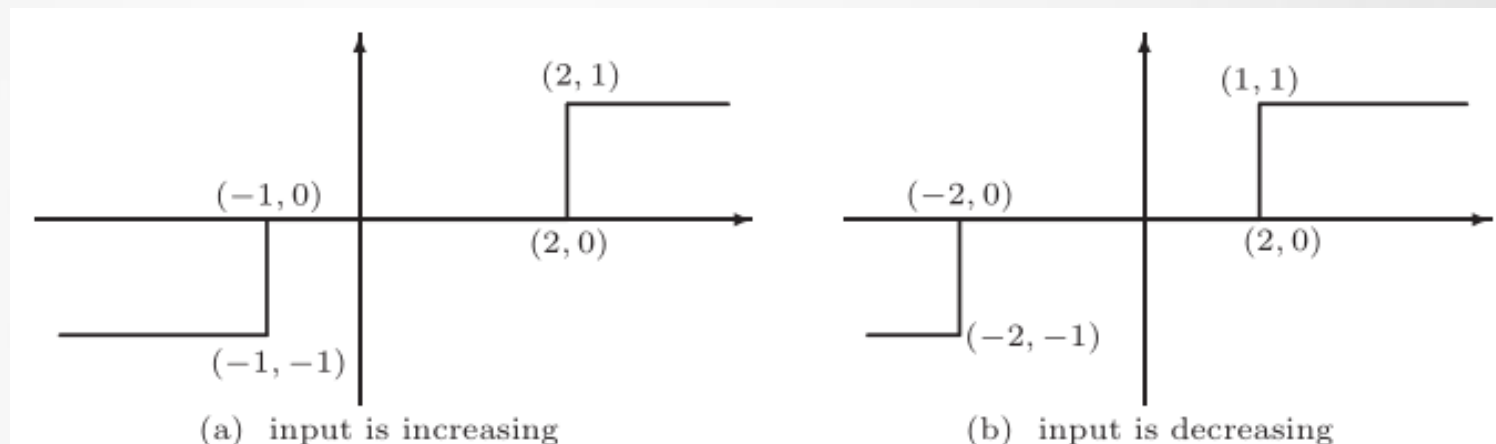
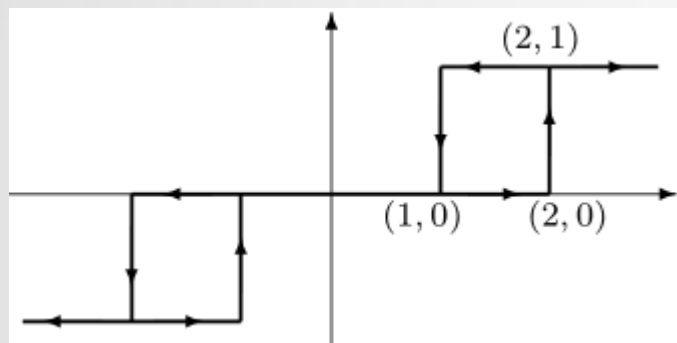


- 如何表示？想法：
 - 分成两个单值非线性
 - 识别输入信号上升或下降：开关、比较



例6-11 多值非线性的实现

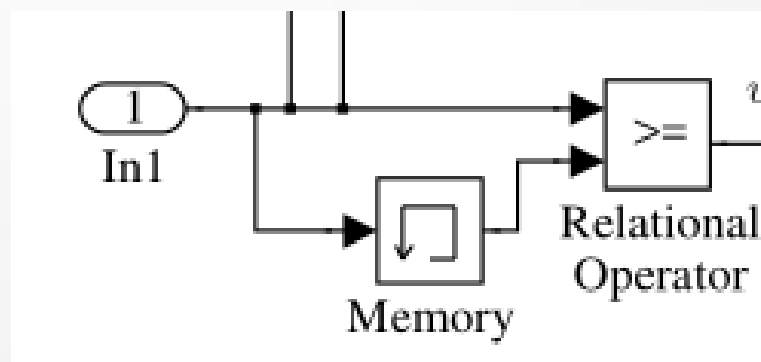
➤ 分解成两个单值非线性



➤ 判定输入信号的增减

➤ 得出前一步信号值

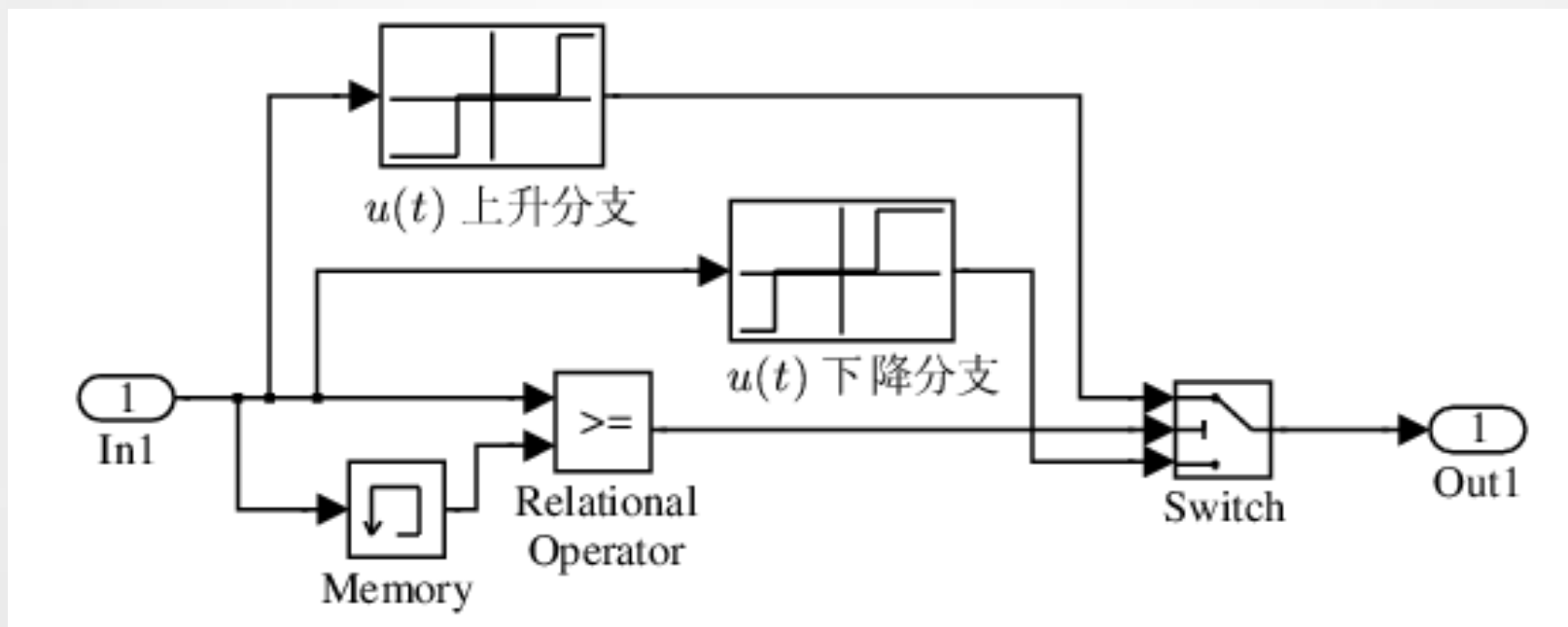
➤ 和当前值比较





Simulink如何实现？

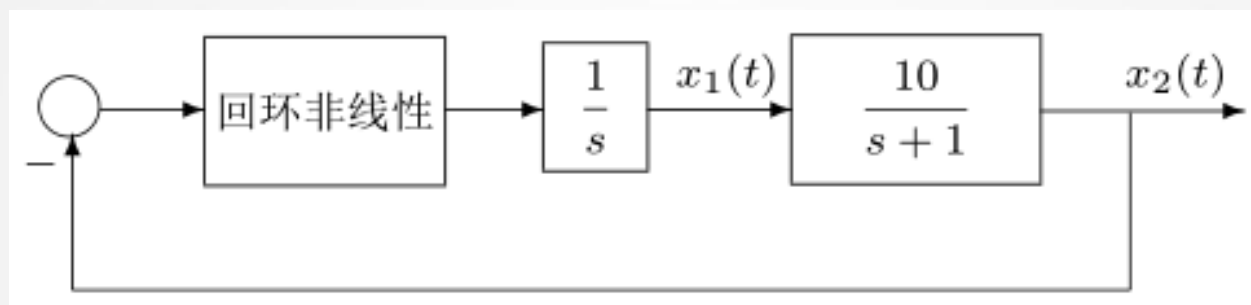
➤ Simulink实现模型



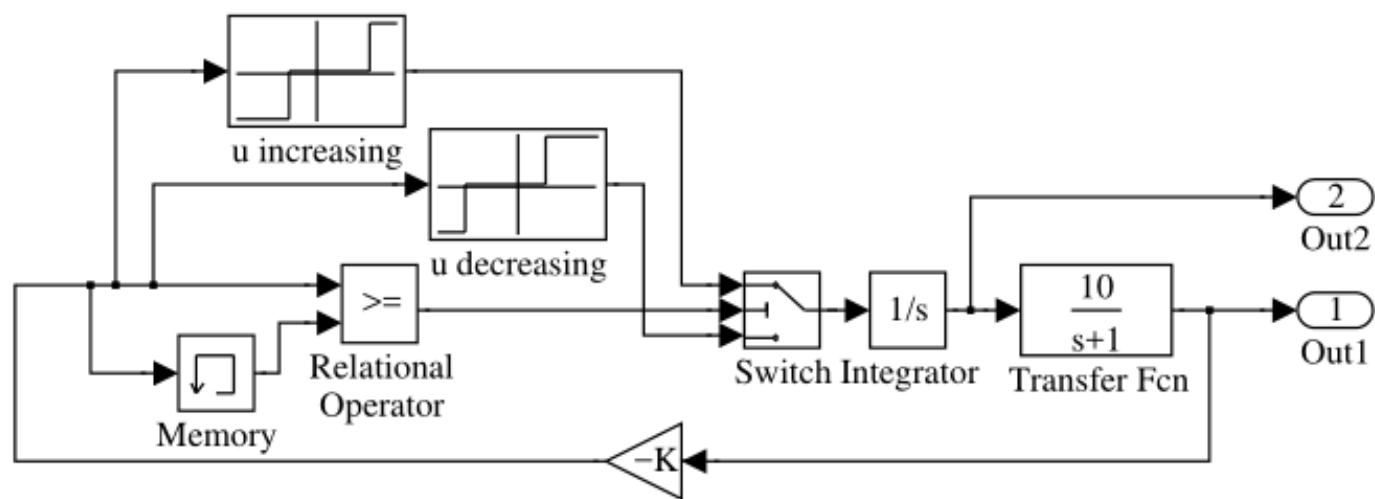


例6-12 非线性系统的极限环

➤ 闭环系统模型



➤ Simulink模型





闭环系统的仿真研究与极限环绘制

➤ Simulink模型名 c6mlimcy.mdl

➤ 非线性系统的仿真

➤ 直接仿真

➤ 终止仿真时间为40



```
>> [t,x,y]=sim('c6mlimcy',40); plot(t,y)
```

➤ 极限环绘制



```
>> plot(y(:,1),y(:,2))
```




非线性模块与系统小结

- Simulink现有的非线性模块有限，需要扩展
- 单值非线性可以直接查表实现
 - 找出转折点，生成 x, y 向量赋给查表模块
 - 直上直下的线如何处理？
- 多值非线性需要拆分成单值的分支
 - 判定信号在上升还是在下降——Memory模块
 - 比较模块、开关模块
- 非线性系统仿真实例——极限环的绘制



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

非线性系统的线性化

Linearization of Nonlinear Systems



主讲：薛定宇教授



非线性系统的线性化

- 线性系统理论成熟
 - 非线性不严重可以用线性系统近似
 - 非线性系统在工作点附近用线性系统近似
- 非线性系统线性化的数学理论与实现
 - 工作点的计算
 - 线性化的数学表示
 - 基于MATLAB的直接计算
- 复杂线性系统模型的化简



平衡点的计算

➤ 非线性状态方程模型

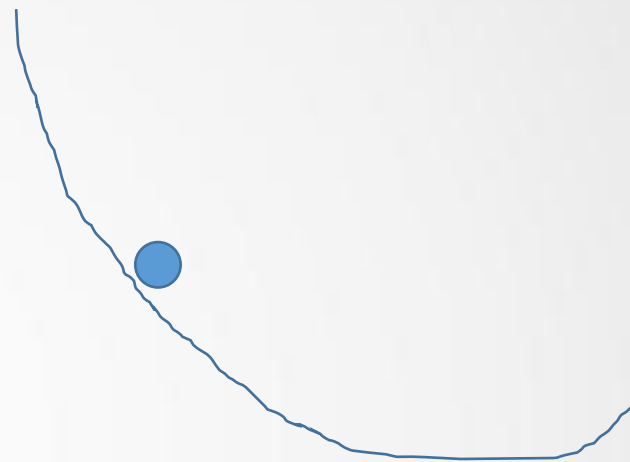
$$\dot{x}_i(t) = f_i(t, x, u), \quad i = 1, 2, \dots, n$$

➤ 平衡点的定义

$$y = f(t, x, u) = 0$$

➤ MATLAB直接求解

$$[x, u, y, x_d] = \text{trim}(\text{model}, x_0, u_0)$$





线性化的数学理论

➤ 状态方程的Taylor级数展开

$$\Delta \dot{x}_i = \sum_{j=1}^n \left. \frac{\partial f_i(t, \mathbf{x}, \mathbf{u})}{\partial x_j} \right|_{\mathbf{x}_0, \mathbf{u}_0} \Delta x_j + \sum_{j=1}^p \left. \frac{\partial f_i(t, \mathbf{x}, \mathbf{u})}{\partial u_j} \right|_{\mathbf{x}_0, \mathbf{u}_0} \Delta u_j$$

$$\mathbf{z}(t) = \Delta \mathbf{x}(t), \quad \mathbf{v}(t) = \Delta \mathbf{u}(t)$$

$$\dot{\mathbf{z}}(t) = \mathbf{A}_1 \mathbf{z}(t) + \mathbf{B}_1 \mathbf{v}(t)$$

$$\mathbf{A}_1 = \begin{bmatrix} \partial f_1 / \partial x_1 & \cdots & \partial f_1 / \partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_n / \partial x_1 & \cdots & \partial f_n / \partial x_n \end{bmatrix}, \quad \mathbf{B}_1 = \begin{bmatrix} \partial f_1 / \partial u_1 & \cdots & \partial f_1 / \partial u_p \\ \vdots & \ddots & \vdots \\ \partial f_n / \partial u_1 & \cdots & \partial f_n / \partial u_p \end{bmatrix}$$



线性化的**MATLAB**求解

- 有了Simulink模型就可以直接线性化

`G=linearize(model,op)`

- 早期版本命令

`[A,B,C,D]=linmod2(model,x0,u0)`

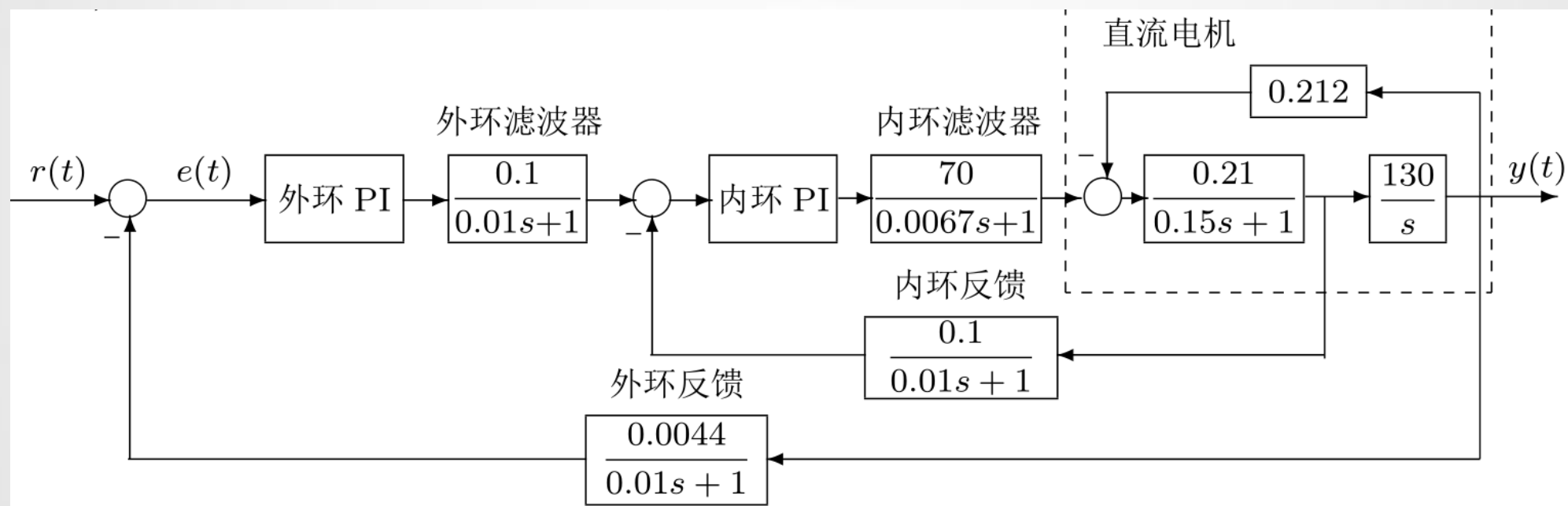
`[A,B,C,D]=linmod(model,x0,u0)`

`[A,B,C,D]=dlinmod(model,x0,u0)`



例6-16 双闭环系统的化简

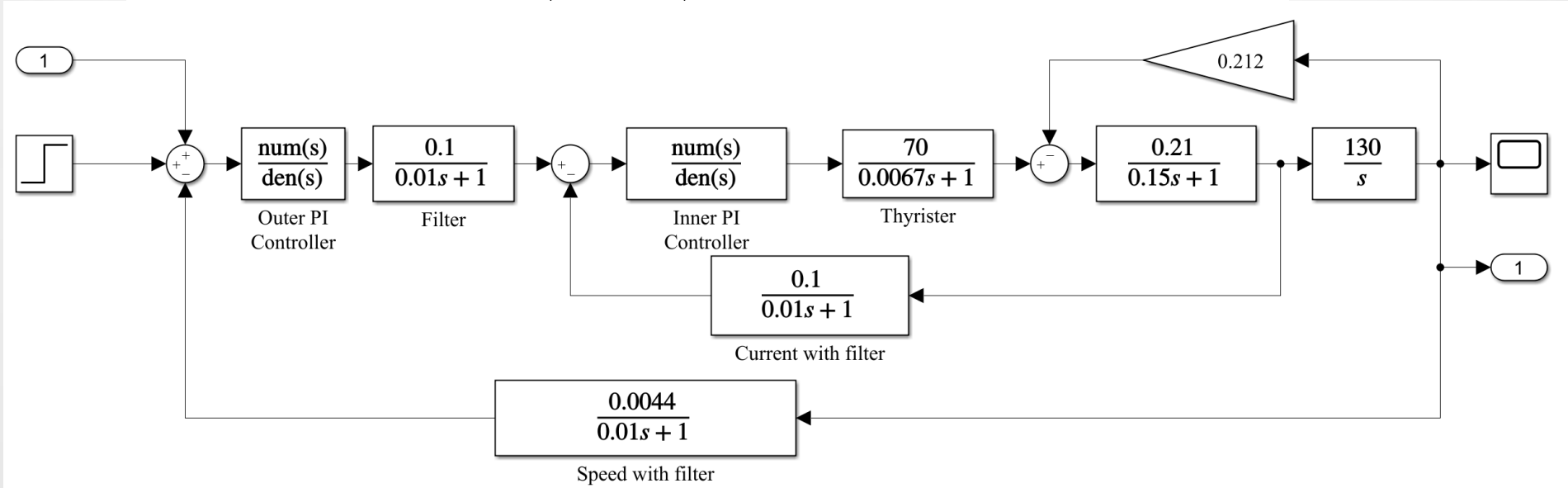
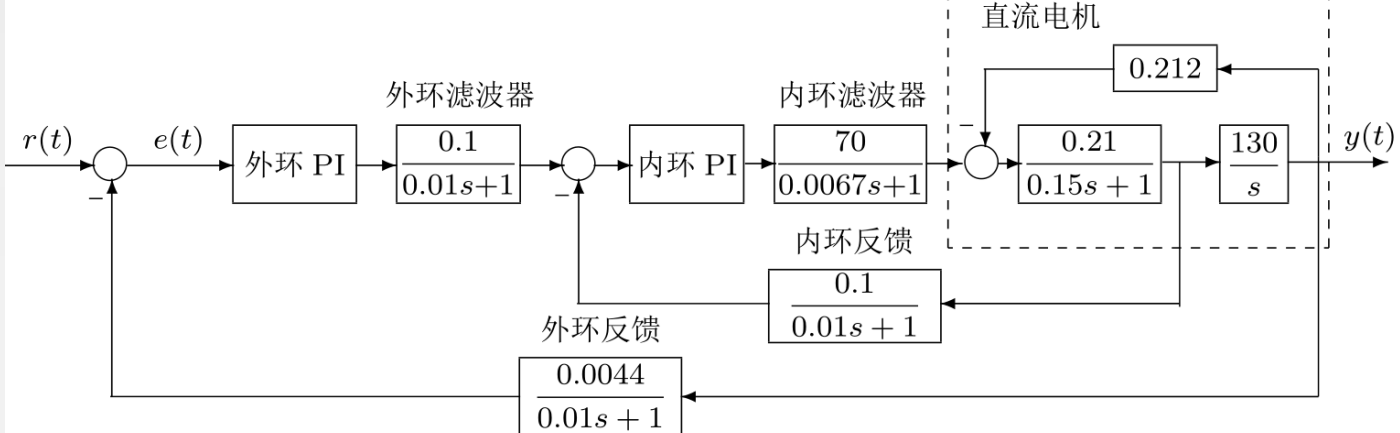
➤ 双闭环系统



$$K_{p1} = 37.9118, K_{i1} = 12.1855, K_{p2} = 10.8489, K_{i2} = 0.9591$$



Simulink建模





线性化与时域响应比较

➤ 线性化

```
 >> Kp1=37.9118; Ki1=12.1855; Kp2=10.8489; Ki2=0.9591;  
      G=linearize('c6mdcm'), G1=zpk(G)
```

```
 >> G2=minreal(G1,1e-1)
```

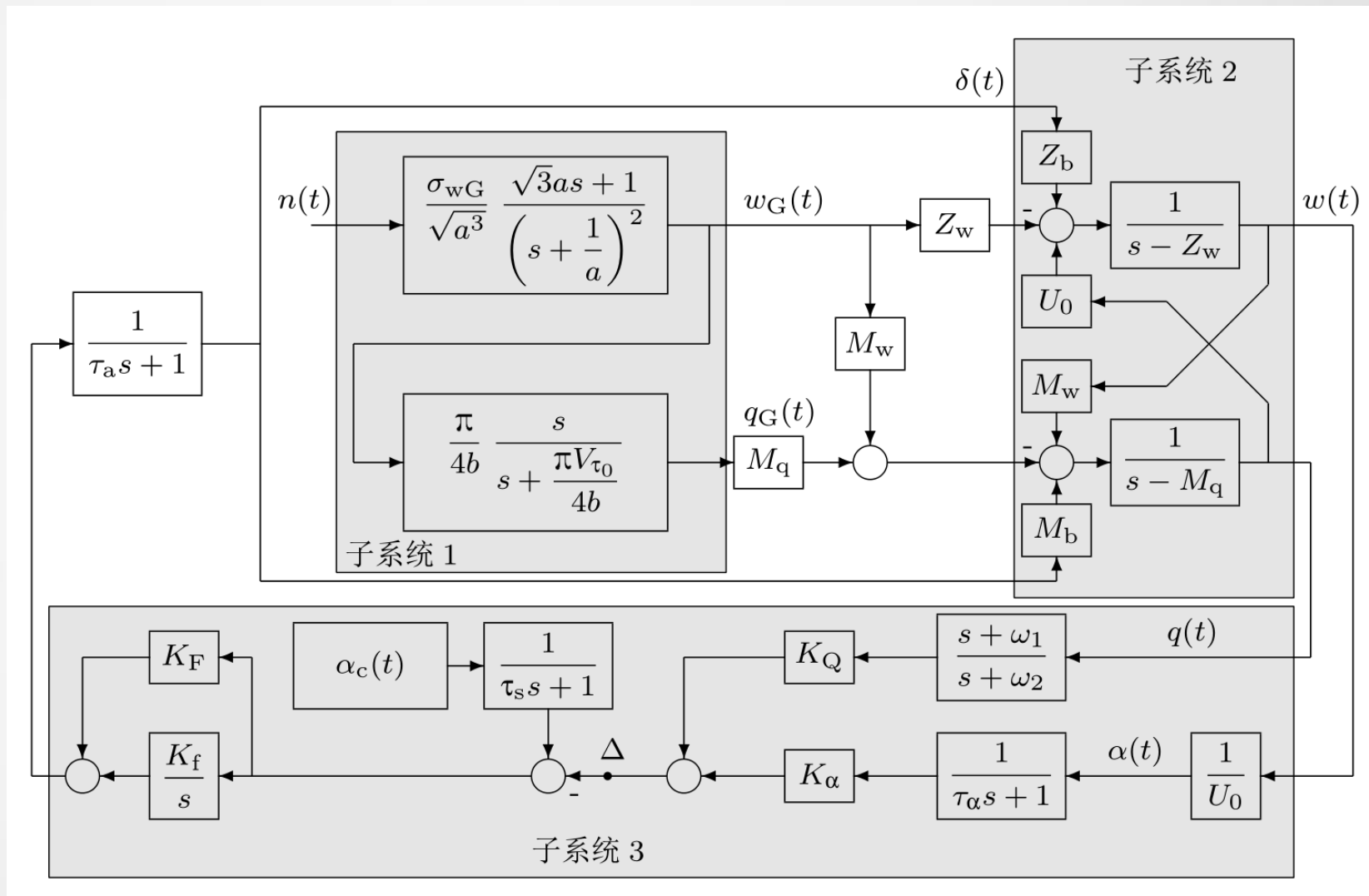
➤ 时域响应比较

```
 >> [t,x,y]=sim('c6mdcm');  
      [y1,t1]=step(G1,0.6); [y2,t2]=step(G2,0.6);  
      plot(t,y,t1,y1,t2,y2)
```



例6-17 F14战斗机的线性化

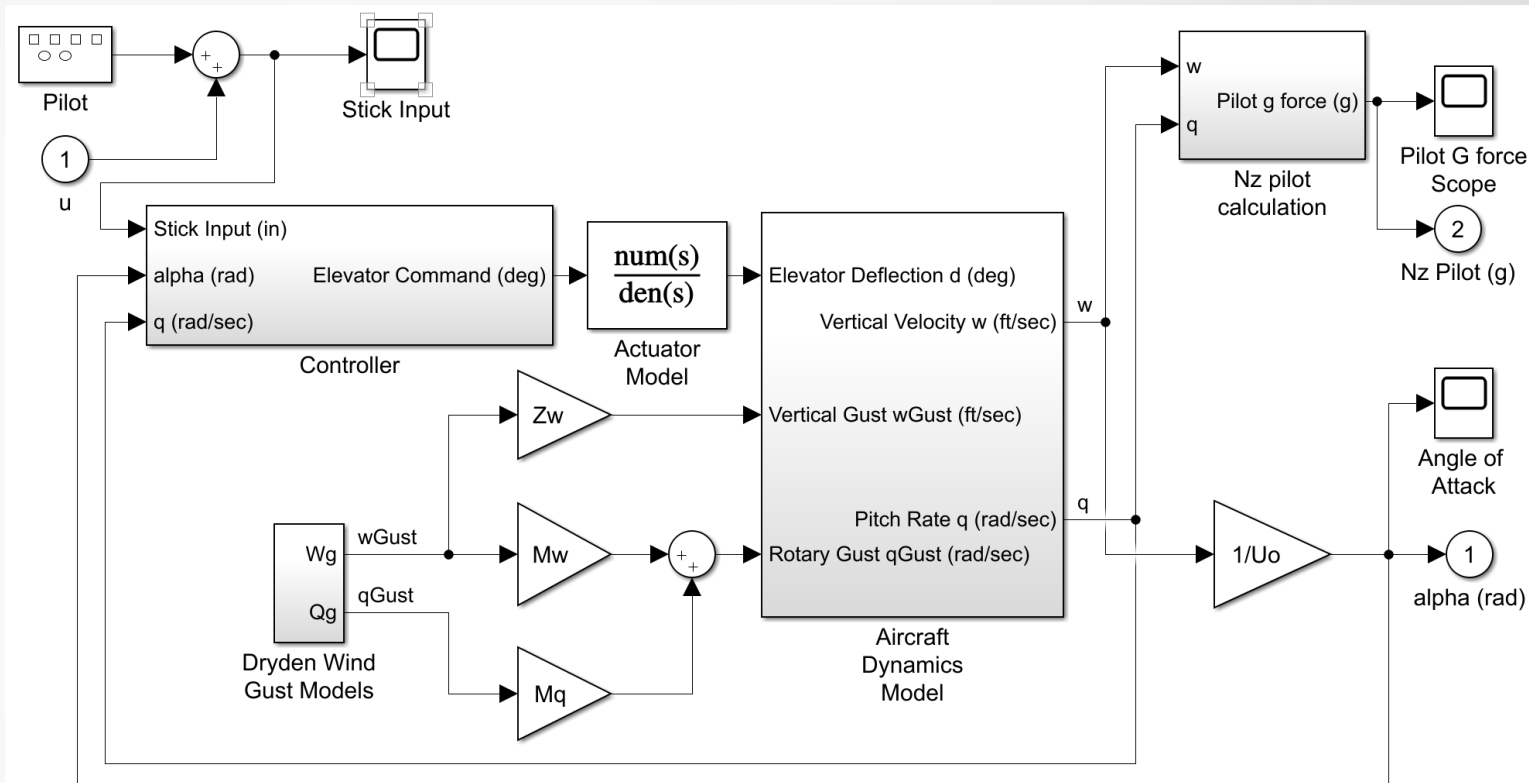
➤ 复杂系统





MATLAB提供了现成的F14 Simulink模型

➤ 仿真模型名称 f14



➤ 线性化命令与结果



```
>> G=linearize('f14'), zpk(G)
```



非线性系统线性化小结

- 线性化的作用
 - 非线性系统 —— trim, linearize
 - 在平衡点附近作线性近似
 - 平衡点的概念与计算
 - 线性化的直接计算
 - 线性系统
 - 线性系统的模型化简



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

子系统 Subsystems



主讲：薛定宇教授



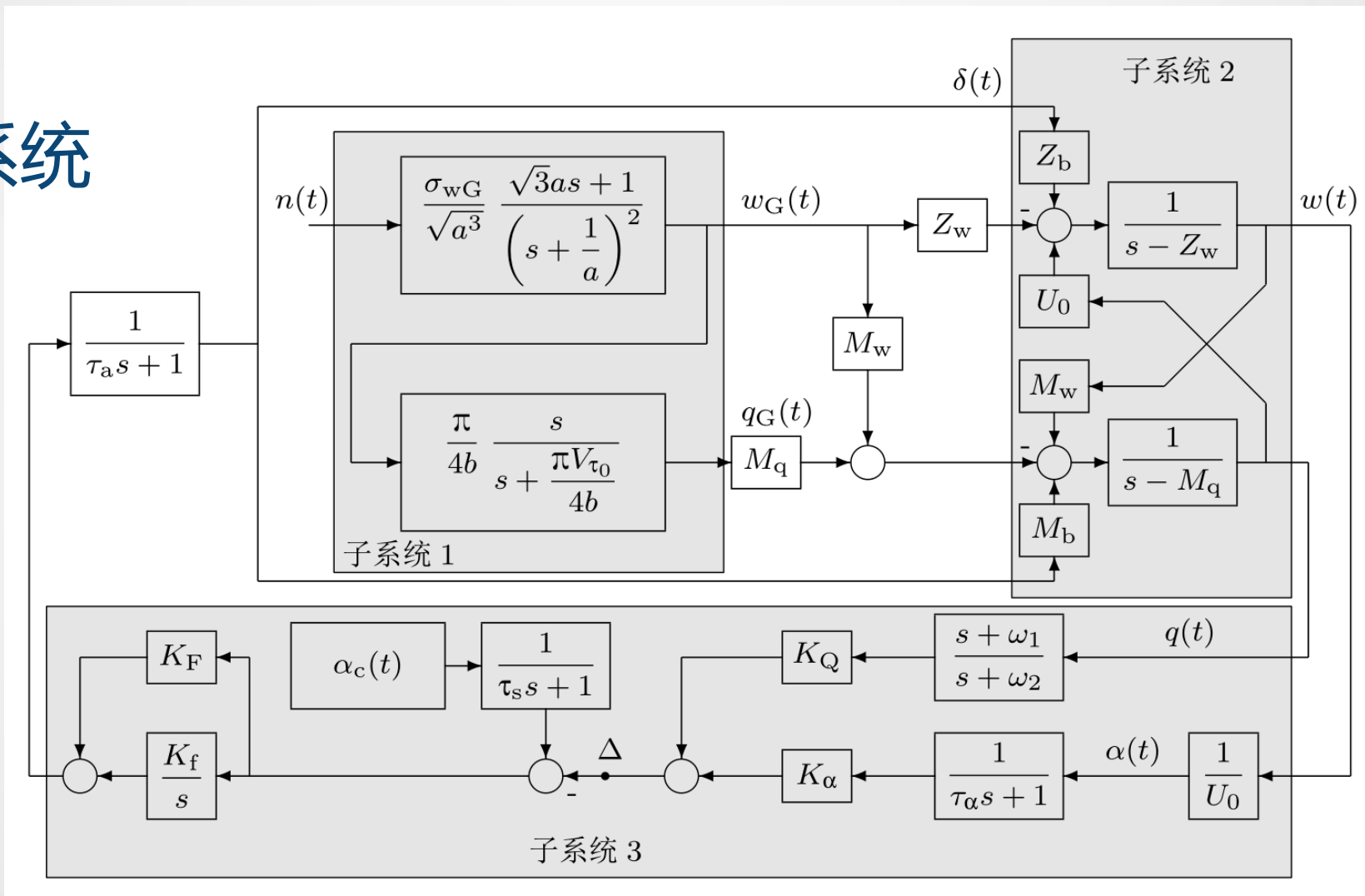
大系统的子系统表示

- 将系统按功能分解成若干相互关联的子系统
 - 子系统的实例 F14
 - 子系统的好处
- 子系统的实现方法



例6-20 F14战斗机的子系统表示

➤ 复杂系统





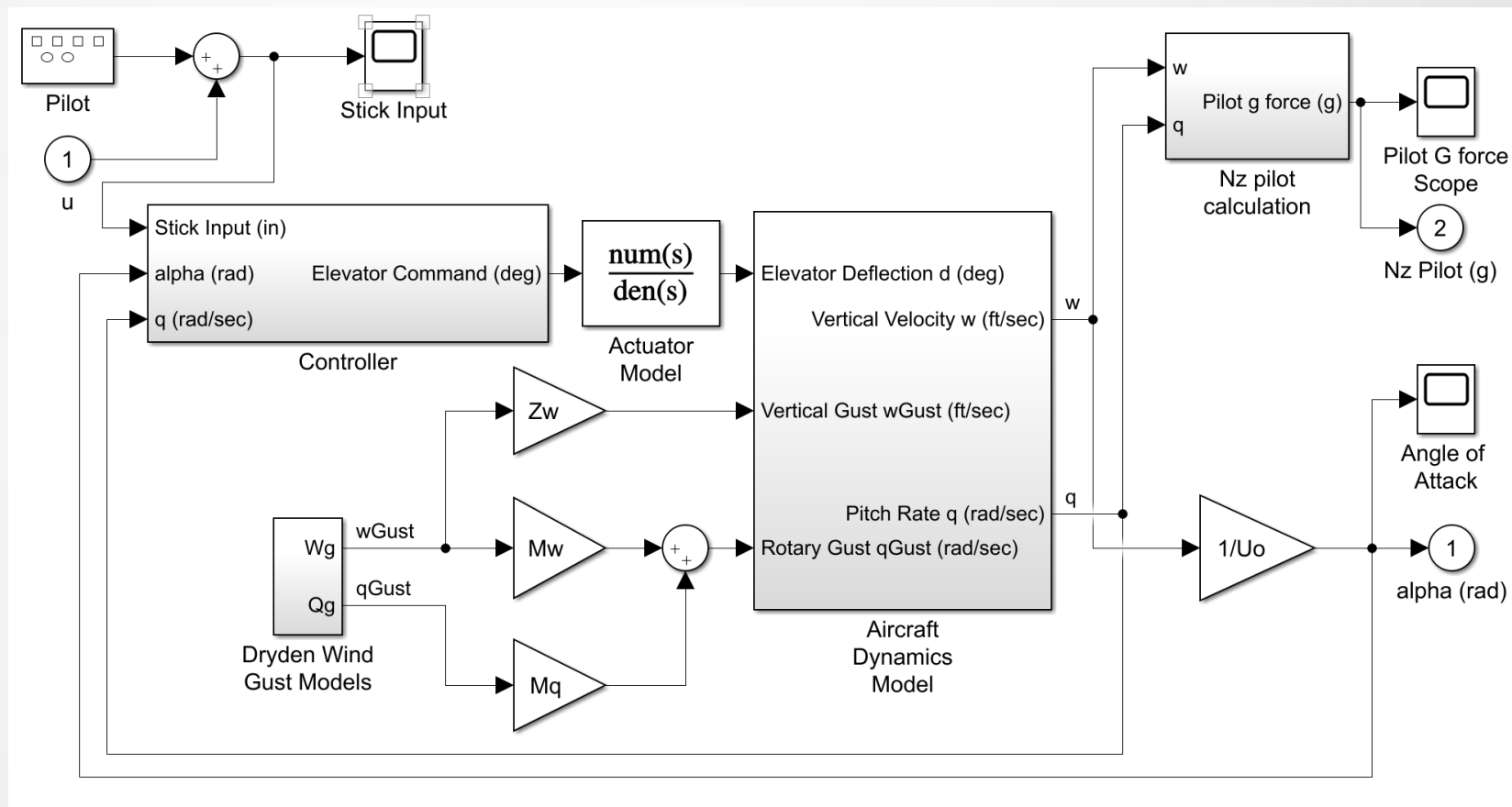
MATLAB提供了现成的F14 Simulink模型

➤ 仿真模型名称 f14

➤ 打开模型

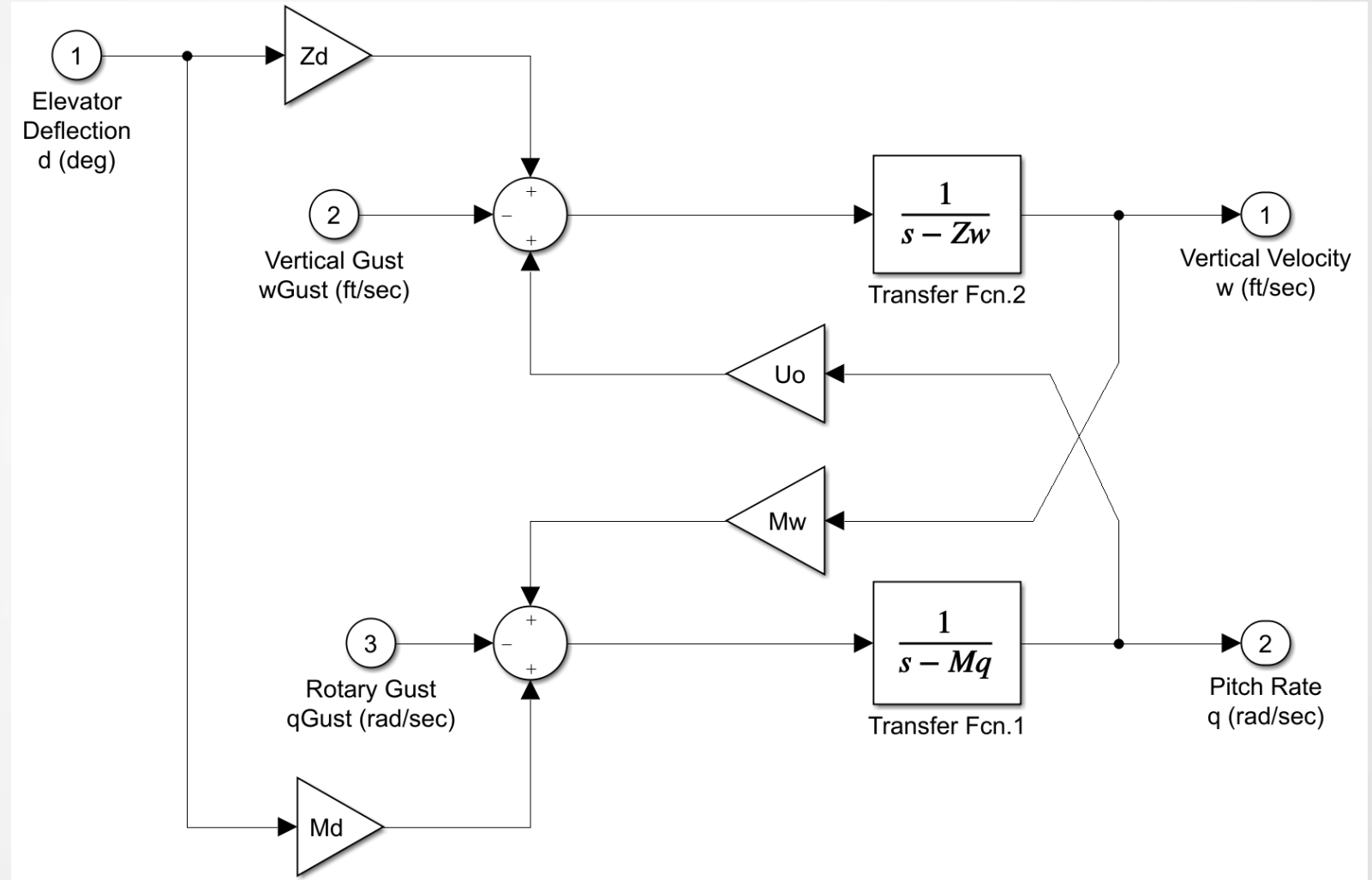
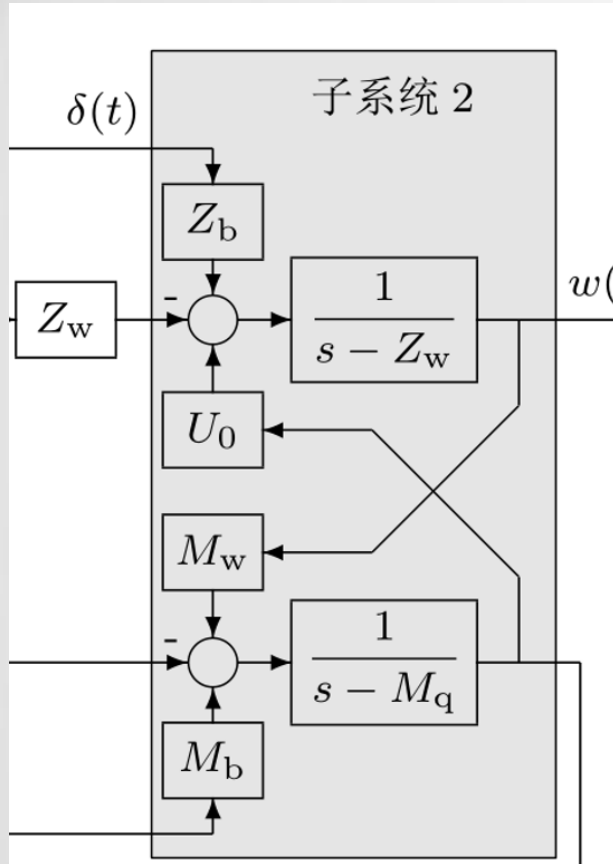


>> f14





子系统模块举例





例6-21 子系统的建立——PID控制器模块

- 新版本有通用PID模块，这里只用于演示
- PID控制器数学模型

$$U(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{s T_d}{1 + s T_d / N} \right) E(s)$$

- 如何建模：比例、积分器、微分器、加法器
 - 连线构造子系统
 - 子系统菜单
- 如何由现有大模型里提取子系统



PID子系统建模

➤ 子系统所需的模块

➤ 输入、输出端子

➤ 比例、积分器、传递函数、加法器

$$U(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{s T_d}{1 + s T_d / N} \right) E(s)$$

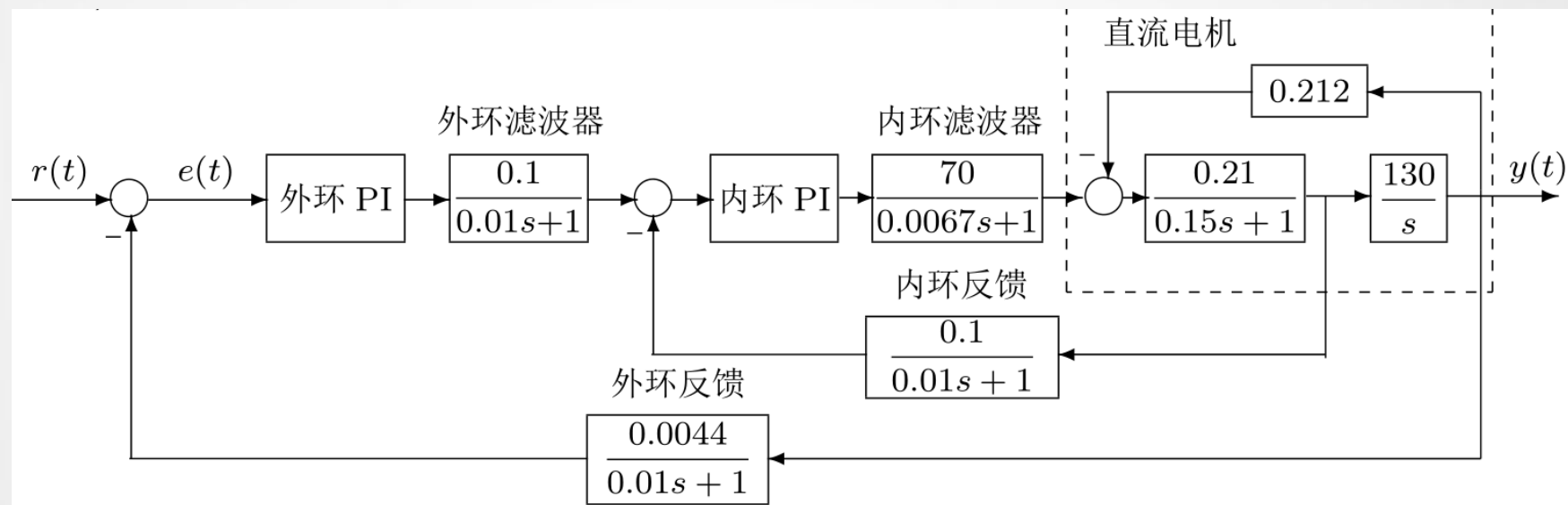
➤ 由底层模块搭建PID控制器模型

➤ PID控制器参数由变量给出



子系统的局限性

➤ 双闭环直流调速系统



➤ 能不能用PID子系统？



子系统小结

- 子系统的作用
 - 将大系统按功能分成相互关联的子系统
 - 化整为零，模型更具结构化，更易于处理
- 子系统构造
 - 以PID控制器为例演示了子系统建模的全过程
- 子系统面临的问题



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

子系统的封装

Subsystems Masking



主讲：薛定宇教授



子系统的封装

- 子系统的局限性
 - 将其变换成相互独立的模块，互不影响
 - 在同一个系统中可以使用若干系统的封装模块
 - 参数可以独立设置
- 封装模块的设计
 - PID控制器为例
 - 图标设计、内部程序处理与设置



例6-23 PID控制器子系统的封装

➤ PID控制器的数学表达式

$$U(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{s T_d}{1 + s T_d / N} \right) E(s)$$

➤ 现成的PID子系统——封装步骤

➤ Edit -> Mask Subsystems

➤ 设计图标

➤ 设计参数对话框——选择变量名

➤ 设计初始化



模块封装的图标设计

➤ 图标处理 (Drawing commands)

➤ 画曲线 (新版本支持后者)



```
plot(cos(0:.1:2*pi),sin(0:.1:2*pi))
```



```
t=0:.1:2*pi; plot(cos(t),sin(t));  
plot(-0.4+0.1*cos(t),0.2+0.1*sin(t)); plot(0.4+0.1*cos(t),0.2+0.1*sin(t));  
t=0:.1:pi; plot(0.6*cos(t),-0.2-0.4*sin(t));
```

➤ 写文字



```
disp('PID\nController')
```

➤ 加图像



```
image(imread('tiantan.jpg'))
```




参数对话框设计

- 4个参数——设计参数对话框
 - K_p —— 比例系数
 - T_i —— 积分系数
 - T_d —— 微分系数
 - N —— 滤波常数
- 对这种简单模块，参数将直接写入模块，无需进一步处理
- PID模块是可重用的



例6-24 双值非线性环节模块封装

➤ 内部结构

➤ c6msin

➤ 端口变换

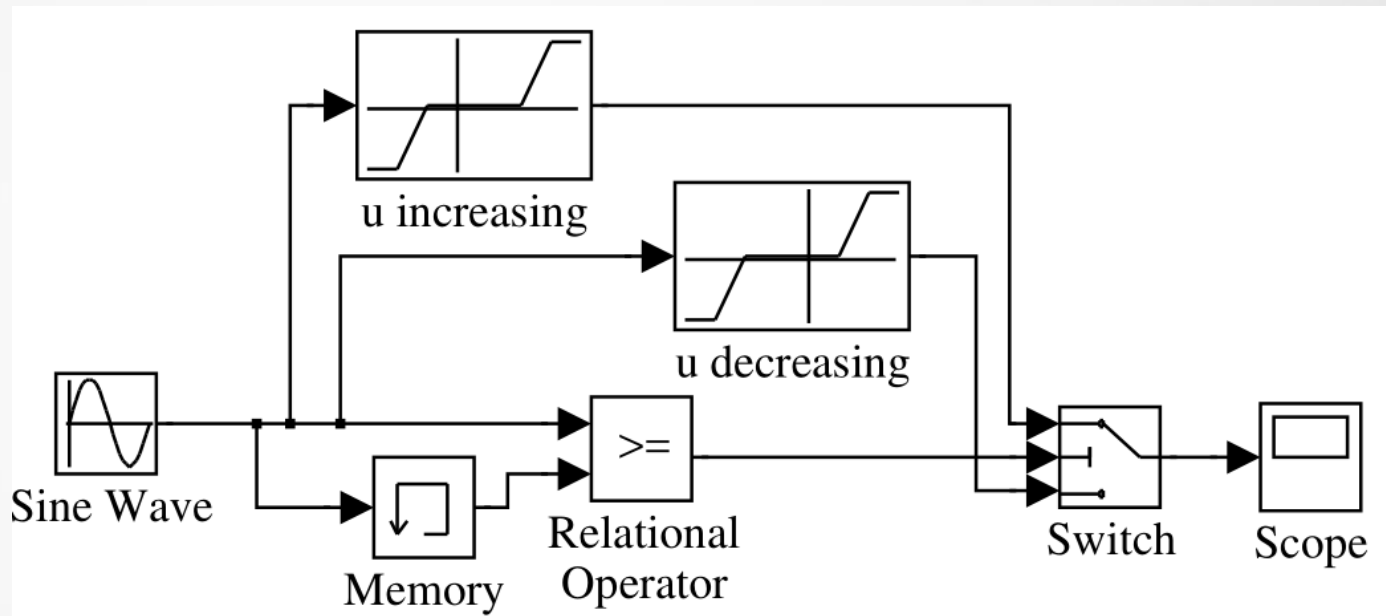
➤ 同时兼顾单值

➤ 模型参数

(xu,yu), (xd,yd)

➤ Parameters栏目

➤ xx、yy（上升段、下降段）——双列矩阵





封装模块设计

➤ 图标设计

➤ 直接由 `xx, yy` 画图



`plot(xx',yy')`

➤ 参数规则

➤ 假设上升段模块使用变量名 `(xu,yu)`

➤ 下降段模块中使用变量名 `(xd,yd)`

➤ 初始化设计

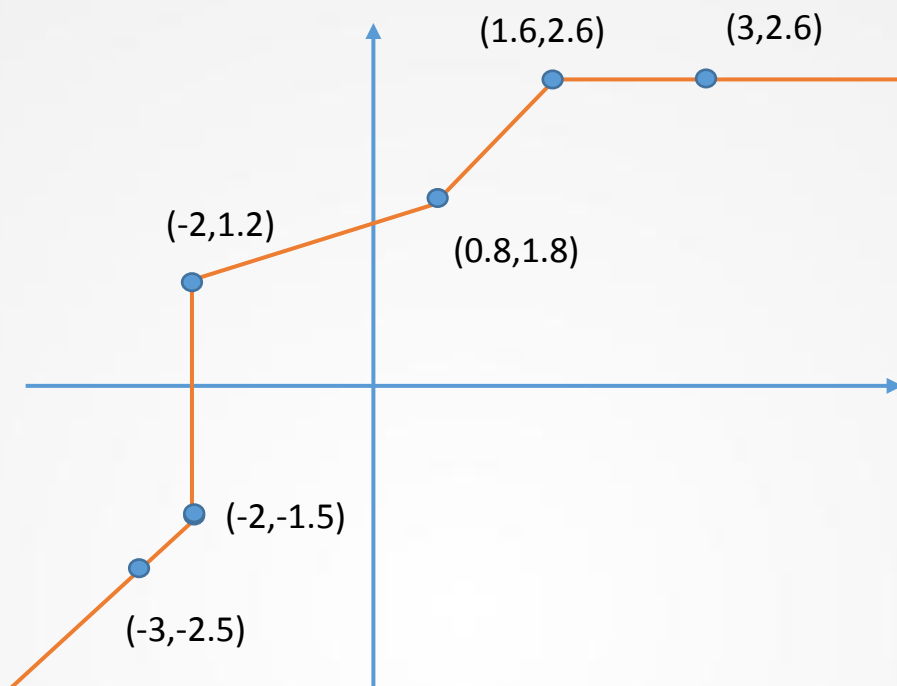


```
if size(yy,1)==1, xx=[xx; xx]; yy=[yy; yy]; end;  
yu=yy(1,:); yd=yy(2,:); xu=xx(1,:); xd=xx(2,:);
```



应用举例

➤ 单值非线性



 `>> xx=[-3,-2,-2+eps,0.8,1.6,3]; yy=[-2.5 -1.5 1.2 1.8 2.6 2.6];`

➤ 双值非线性

 `[-3,-2,-1, 2, 3, 4; -4,-3,-2, 1, 2, 3]`

 `[-1,-1,0,0,1,1; -1,-1,0,0,1,1]`



子系统封装小结

- 对子系统操作可以封装
 - 图标设计
 - 内部模型与代码处理



控制系统仿真与 CAD

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

S-函数编程 (上)

S-Function Programming (I)



主讲：薛定宇教授



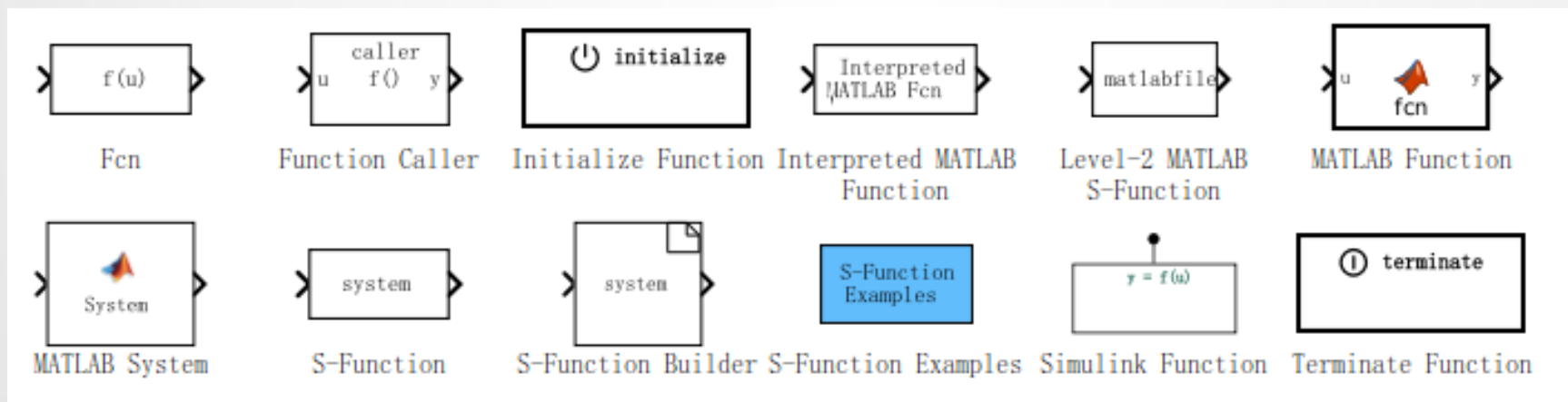
M-函数与S-函数

- 为什么需要M-或S-函数
 - 底层模块搭建繁琐
 - MATLAB语言、C、Ada等
- M-函数与S-函数
 - M-模块：静态关系 $y = f(u)$
 - S-函数——系统函数
 - 动态关系——状态方程、任意复杂系统
- S-函数的编程结构与框架
- S-函数举例



自定义函数模块组

- User-defined Functions
 - 弥补底层模块搭建的不足
 - 两类模块——静态与动态



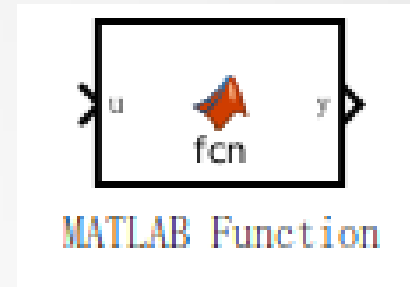
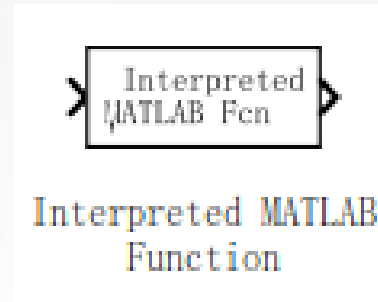


M-函数

- 描述函数的静态关系
- M-函数模块
 - User-Defined Functions
- 饱和和非线性的M-描述编程

```
function y=satur_non(x)
if abs(x)>=3, y=2*sign(x); else, y=2/3*x; end
```

- M-函数的局限性
 - 不能带有附加参数：宽度、斜率





为什么需要S-函数

➤ 例6-23 韩京清研究员提出的微分-跟踪器

$$\begin{cases} x_1(k+1) = x_1(k) + Tx_2(k) \\ x_2(k+1) = x_2(k) + T\text{fst}(x_1(k), x_2(k), u(k), r, h) \end{cases}$$

$$\delta = rh, \quad \delta_0 = \delta h, \quad b = x_1 - u + hx_2, \quad a_0 = \sqrt{\delta^2 + 8r|b|}$$

$$a = \begin{cases} x_2 + b/h, & |b| \leq \delta_0 \\ x_2 + 0.5(a_0 - \delta)\text{sign}(b), & |b| > \delta_0 \end{cases}$$

$$\text{fst} = \begin{cases} -ra/\delta, & |a| \leq \delta \\ -r\text{sign}(a), & |a| > \delta \end{cases}$$



S-函数(状态方程)的基本结构

➤ S-函数入口语句

```
function [sys,x0,str,ts]=fun(t,x,u,flag,p1,p2, $\cdots$ )
```

➤ S-函数的基本框架

```
switch flag
case 0, [sys,x0,str,ts] = mdlInitializeSizes(T);
case 1, sys = mdlDeravativess(x,u,r,h,T);
case 2, sys = mdlUpdates(x,u,r,h,T);
case 3, sys = mdlOutputs(x);
end
```



S-函数的执行过程

- flag变量的自动取值
 - 初始化：flag=0, 输入、输出路数、状态数等
 - flag=3, 计算输出信号
 - flag=1, 2, 分别更新状态：连续、离散状态方程
- S-函数用模块表示

```
switch flag
case 0, [sys,x0,str,ts] = mdlInitializeSizes(T);
case 1, sys = mdlDeravitivess(x,u,r,h,T);
case 2, sys = mdlUpdates(x,u,r,h,T);
case 3, sys = mdlOutputs(x);
end
```



S-函数的初始化

- 由simsizes读入模板，成员变量
 - .NumContStates，S-函数中连续状态的个数
 - .NumDiscStates，表示离散状态的个数
 - .NumInputs，模块的输入路数
 - .NumOutputs，表示模块的输出路数
 - .DirFeedthrough，输出方程是否显含 u
 - .NumSampleTimes，采样周期的个数
- 由simsizes返回
- 计算初值、采样周期等



其他函数结构

➤ 系统的状态 $\mathbf{x} = [\mathbf{x}_c, \mathbf{x}_d]^T$

➤ 连续状态更新 $\dot{\mathbf{x}}_c(t) = \mathbf{F}(t, \mathbf{x}, \mathbf{u})$

```
function sys = mdlDeravativivess(x,u,r,h,T);
```

➤ 离散状态更新 $\mathbf{x}_d(k+1) = \mathbf{G}(t, \mathbf{x}(k), \mathbf{u}(k))$

```
function sys = mdlUpdates(x,u,r,h,T);
```

➤ 输出计算 $\mathbf{y}(t) = \mathbf{H}(t, \mathbf{x}(t), \mathbf{u}(t))$

```
function sys = mdlOutputs(x);
```



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

S-函数编程 (下)

S-Function Programming (II)



主讲：薛定宇教授



例6-24 阶梯信号发生器

➤ 阶梯信号发生器

➤ 静态函数

➤ M函数不支持附加变量

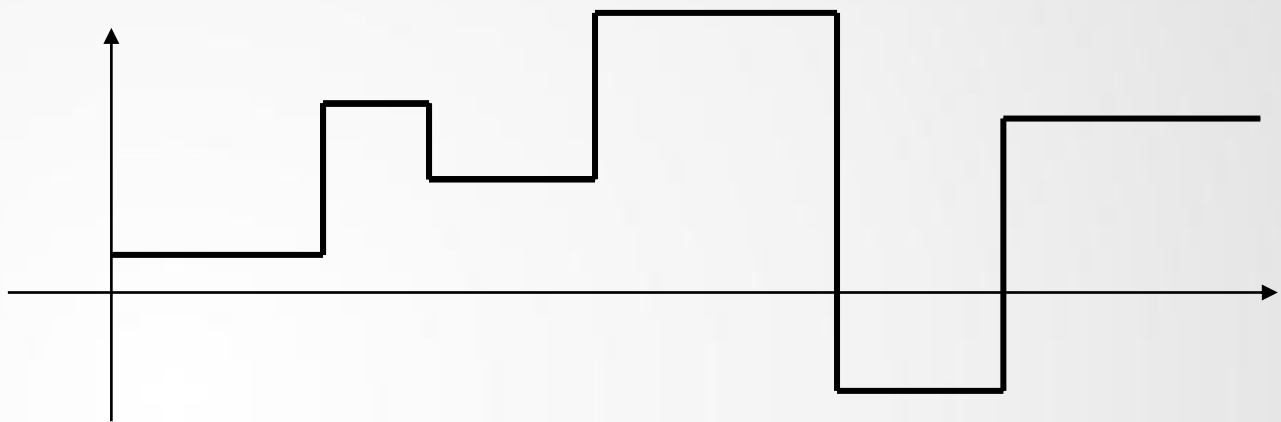
➤ 只能编写S-函数

➤ 转折点坐标

$$\text{tTime}=[t_1, t_2, \dots, t_N], \text{yStep}=[y_1, y_2, \dots, y_N]$$

➤ 输入、输出、状态个数、输入输出关系

➤ 附加参数





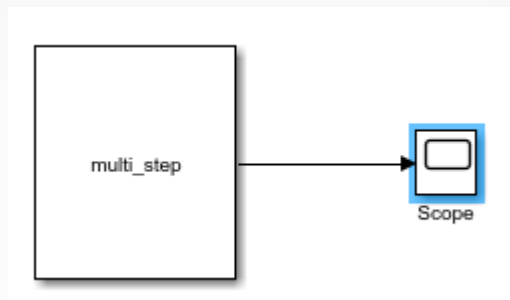
S-函数编写清单

```
function [sys,x0,str,ts]=multi_step(t,x,u,flag,tTime,yStep)
switch flag,
    case 0, [sys,x0,str,ts] = mdlInitializeSizes;
    case 3, sys = mdlOutputs(t,tTime,yStep);
    case {1, 2, 4, 9}, sys = [];
    otherwise, error(['Unhandled flag = ',num2str(flag)]);
end;
function [sys,x0,str,ts] = mdlInitializeSizes
sizes = simsizes; sizes.NumContStates=0;
sizes.NumDiscStates=0; sizes.NumOutputs = 1;
sizes.NumInputs = 0; sizes.DirFeedthrough = 0;
sizes.NumSampleTimes = 1; sys = simsizes(sizes);
x0 = []; str = []; ts = [0 0];
function sys = mdlOutputs(t,tTime,yStep)
i=find(tTime<=t); sys=yStep(i(end));
```



S-函数的使用方法

- 打开模型窗口
- c6mmstep.slx
- 数据点



$x=[0,1,3,5,6,8]; y=[1,-3,4,6,2,6];$



例6-25 微分跟踪器编程实现

➤ 韩京清研究员的微分-跟踪器

$$\begin{cases} x_1(k+1) = x_1(k) + Tx_2(k) \\ x_2(k+1) = x_2(k) + T\text{fst}(x_1(k), x_2(k), u(k), r, h) \end{cases}$$

$$\delta = rh, \quad \delta_0 = \delta h, \quad b = x_1 - u + hx_2, \quad a_0 = \sqrt{\delta^2 + 8r|b|}$$

$$a = \begin{cases} x_2 + b/h, & |b| \leq \delta_0 \\ x_2 + 0.5(a_0 - \delta)\text{sign}(b), & |b| > \delta_0 \end{cases} \quad \text{fst} = \begin{cases} -ra/\delta, & |a| \leq \delta \\ -r\text{sign}(a), & |a| > \delta \end{cases}$$

➤ S函数准备

➤ 模块的输出方程 $y = x$

➤ 状态个数、输入输出路数、附加变量 (r, h, T)



主程序的设计

➤ 主函数

```
function [sys,x0,str,ts]=han_td(t,x,u,flag,r,h,T)
switch flag
case 0, [sys,x0,str,ts] = mdlInitializeSizes(T);
case 2, sys = mdlUpdates(x,u,r,h,T);
case 3, sys = mdlOutputs(x);
case {1, 4, 9}, sys = [];
otherwise, error(['Unhandled flag = ',num2str(flag)]);
end;
```

➤ 输出方程

```
function sys = mdlOutputs(x), sys=x;
```



初始化函数编写

```
function [sys,x0,str,ts] = mdlInitializeSizes(T)
sizes = simsizes;
sizes.NumContStates = 0;
sizes.NumDiscStates = 2;
sizes.NumOutputs = 2;
sizes.NumInputs = 1;
sizes.DirFeedthrough = 0;
sizes.NumSampleTimes = 1;
sys = simsizes(sizes);
x0 = [0; 0];
str = [];
ts = [T 0];
```



离散状态更新程序

- 没有连续状态，无需编程
- 离散状态更新

$$\begin{cases} x_1(k+1) = x_1(k) + T x_2(k) \\ x_2(k+1) = x_2(k) + T \text{fst}(x_1(k), x_2(k), u(k), r, h) \end{cases}$$

- 支持函数

```
function sys = mdlUpdates(x,u,r,h,T)
sys(1,1)=x(1)+T*x(2);
sys(2,1)=x(2)+T*fst2(x,u,r,h);
```





支持函数编写

➤ 数学公式

$$\delta = rh, \quad \delta_0 = \delta h, \quad b = x_1 - u + hx_2, \quad a_0 = \sqrt{\delta^2 + 8r|b|}$$

$$a = \begin{cases} x_2 + b/h, & |b| \leq \delta_0 \\ x_2 + 0.5(a_0 - \delta)\text{sign}(b), & |b| > \delta_0 \end{cases} \quad \text{fst} = \begin{cases} -ra/\delta, & |a| \leq \delta \\ -r\text{sign}(a), & |a| > \delta \end{cases}$$

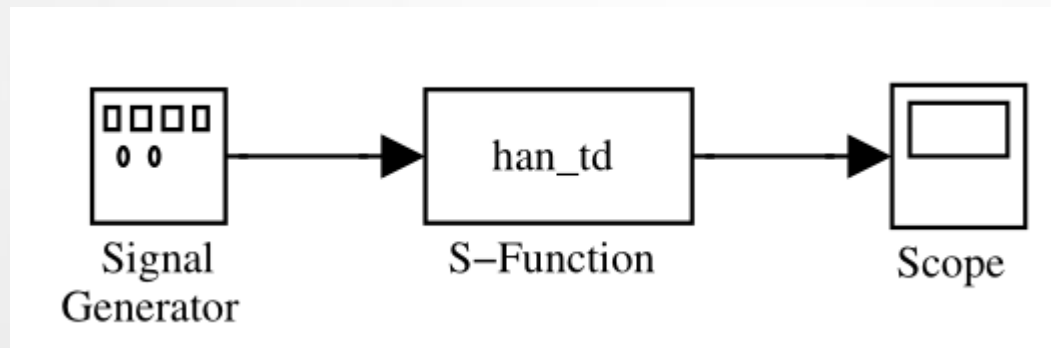
➤ 程序实现

```
function f=fst2(x,u,r,h)
delta=r*h; delta0=delta*h; b=x(1)-u+h*x(2);
a0=sqrt(delta*delta+8*r*abs(b));
a=x(2)+b/h*(abs(b)<=delta0)+0.5*(a0-delta)*sign(b)*(abs(b)>delta0);
f=-r*a/delta*(abs(a)<=delta)-r*sign(a)*(abs(a)>delta);
```



模块的使用举例

- 用正弦信号去激励S-函数模块
- 得出跟踪信号与其导数信号
- 仿真模型 c6msf2





S-函数小结

- 为什么需要M-函数与S-函数
- M-函数的编程实现与局限性
- S-函数的结构与执行机制
 - Switch case标准框架 —— 适用于所有S-函数
 - 初始化 flag=0
 - 计算模块的输出, flag=3
 - 状态更新——离散与连续状态, flag=1,2





国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第六章 非线性系统的建模与仿真

多领域物理建模概述

Introduction to Multi-domain Physical Modeling



主讲：薛定宇教授



多领域物理建模

- 传统底层建模方法的局限性
- 什么是多领域(multi-domain)物理建模
 - 领域——机械、电力、电子、控制
 - 物理建模——按搭建硬件系统的方式组建仿真模型
 - 为什么需要物理建模？
- 多领域物理建模简介
- 多领域物理建模的工具与实例



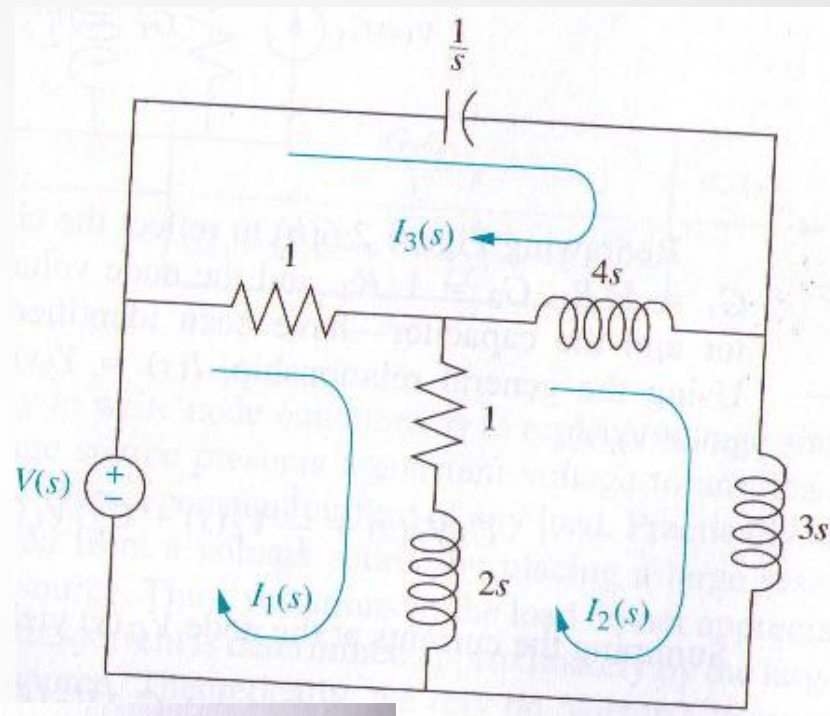
例6-22 电路的建模——传统方法

➤ 电路图举例

(扫描自Nise.

Control Systems Engineering)

➤ 建模



$$\begin{aligned} &+(2s+2)I_1(s) - (2s+1)I_2(s) - I_3(s) = V(s) \\ &-(2s+1)I_1(s) + (9s+1)I_2(s) - 4sI_3(s) = 0 \\ &-I_1(s) - 4sI_2(s) + \left(4s+1+\frac{1}{s}\right)I_3(s) = 0 \end{aligned}$$



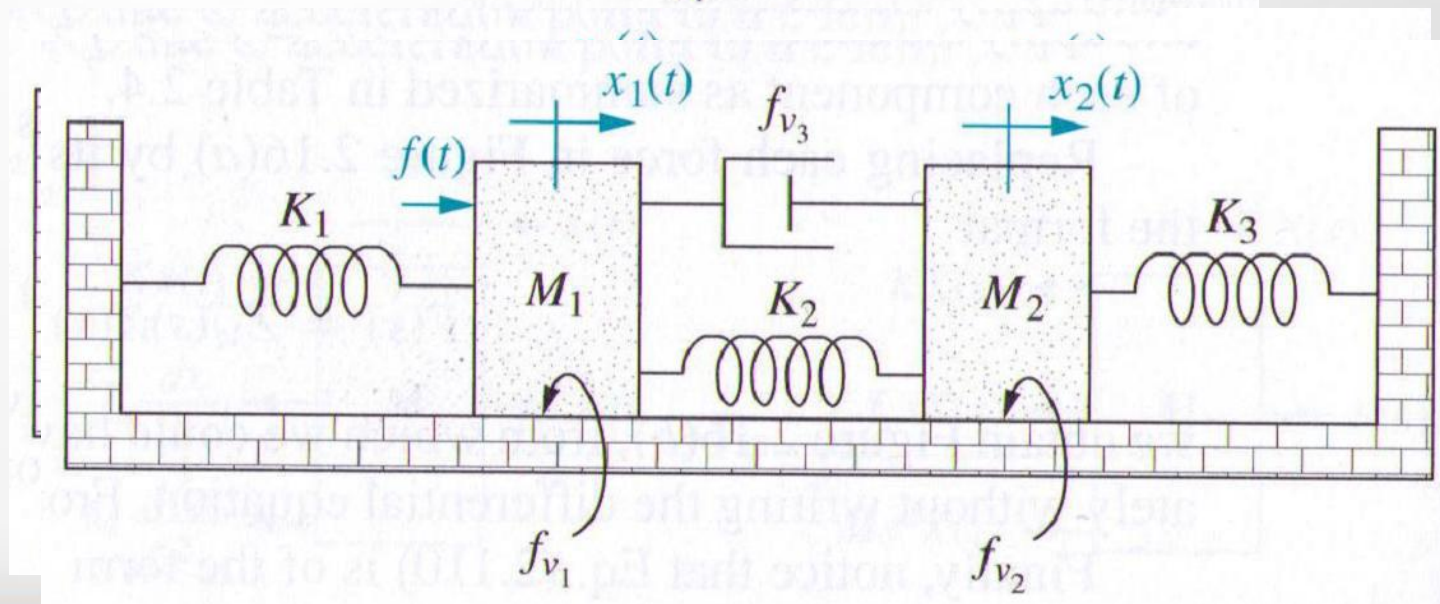
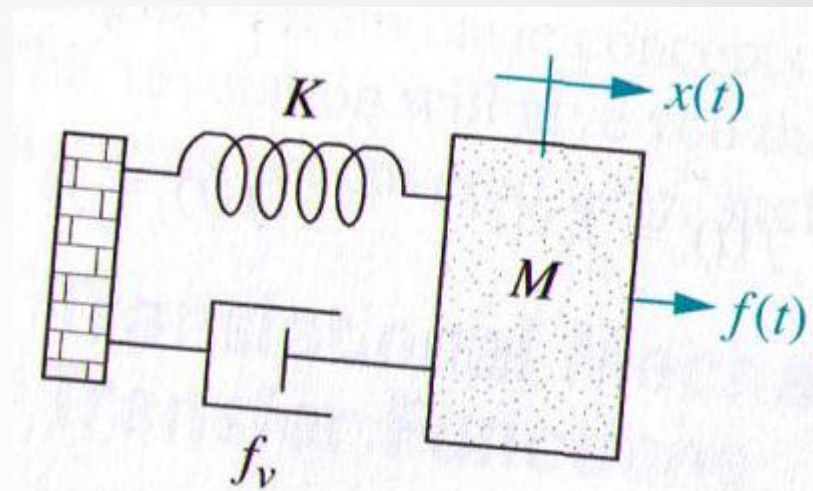
例6-23 力学系统建模——传统方法

➤ 力学系统实例

➤ Newton第二定律 $F = ma$

$$Mx''(t) + f_v x'(t) + Kx(t) = f(t)$$

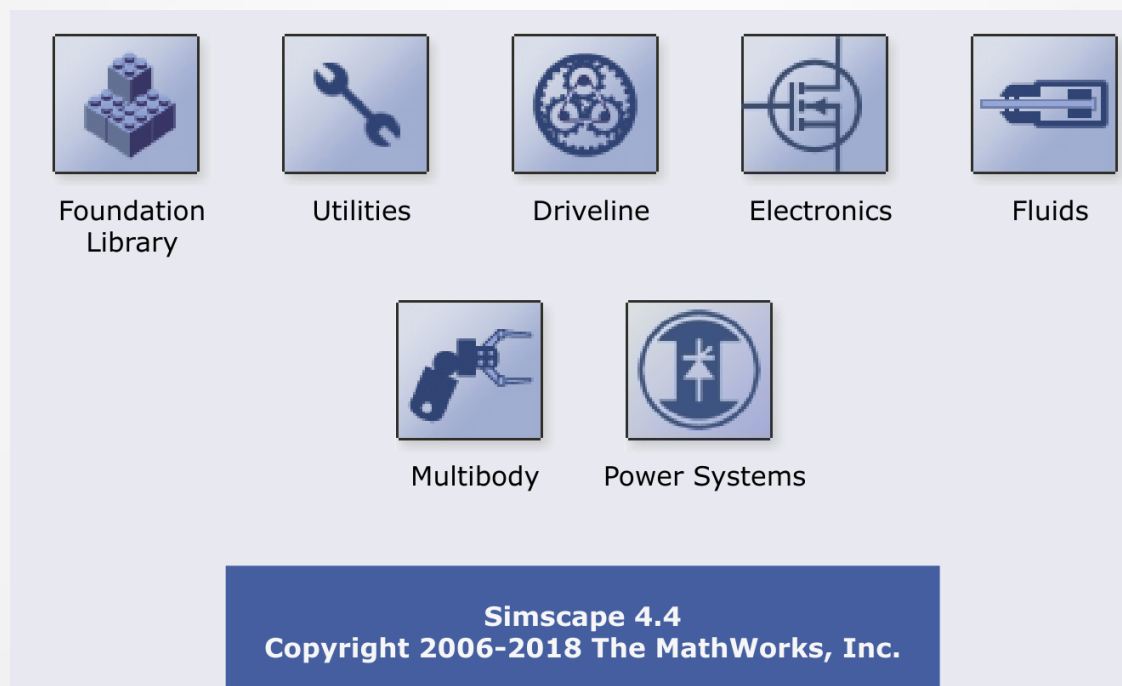
➤ 问题：多模块怎么办？





Simscape模块集

➤ 输入simscape命令打开工具





Foundation Library

Electrical

Gas

Hydraulic

Magnetic

Mechanical

Moist Air

Physical Signals

Thermal

Thermal Liquid

Two-Phase Fluid

Foundation Library
Copyright 2005-2018 The MathWorks, Inc.



机械与力学系统模块

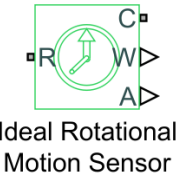
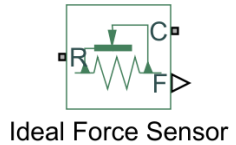
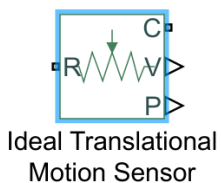
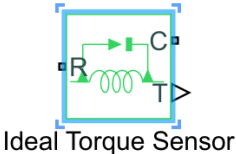
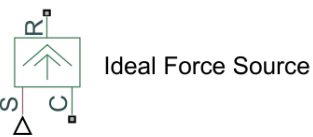
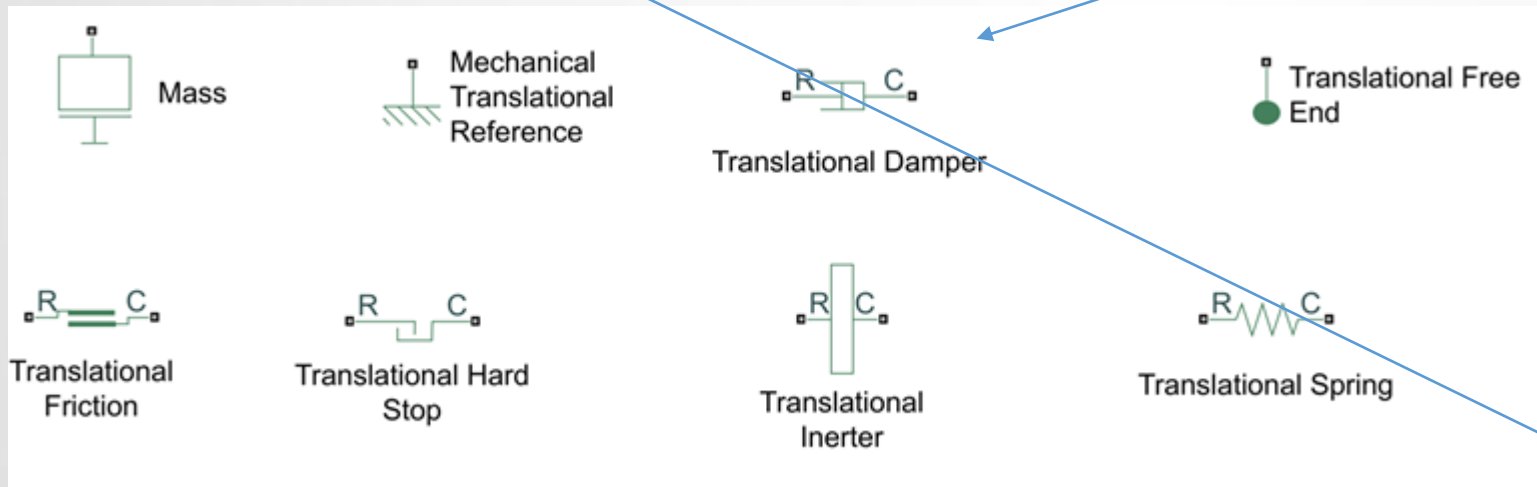
Mechanical Sensors

Mechanical Sources

Mechanisms

Rotational Elements

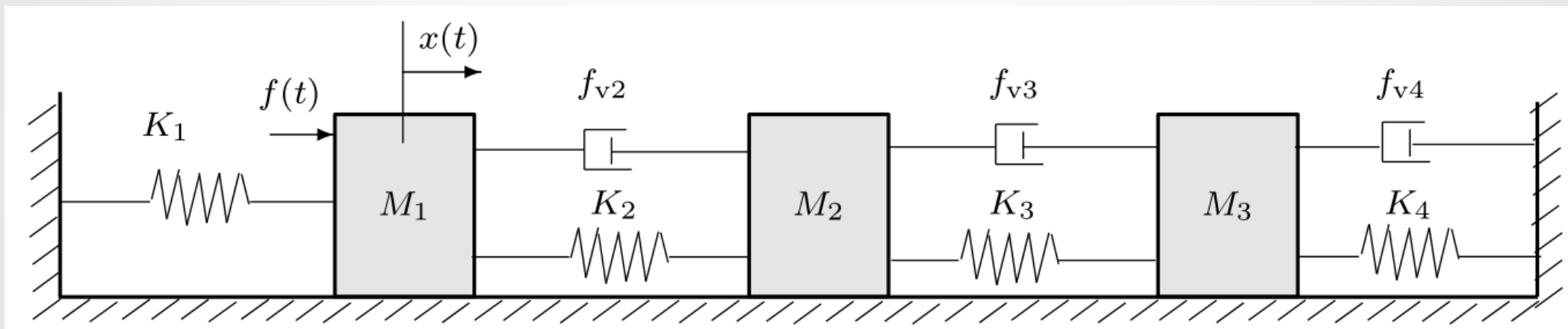
Translational Elements





例6-24 力学系统的实际建模

➤ 系统框图

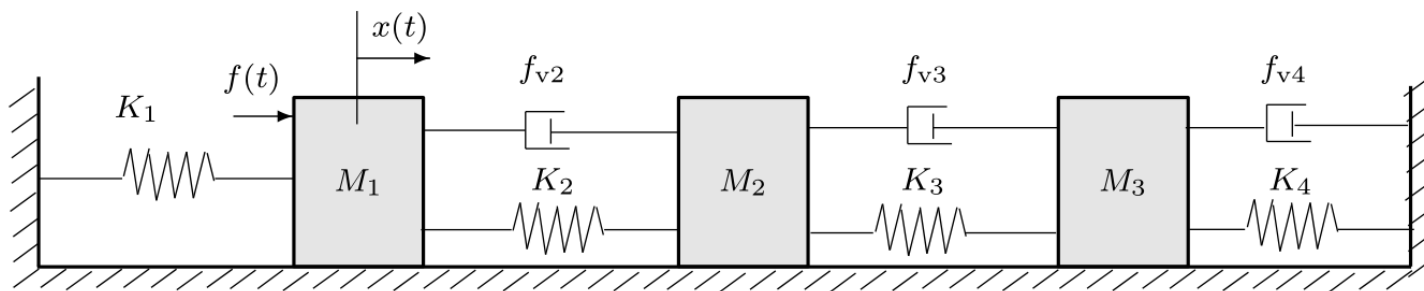


➤ 系统部件

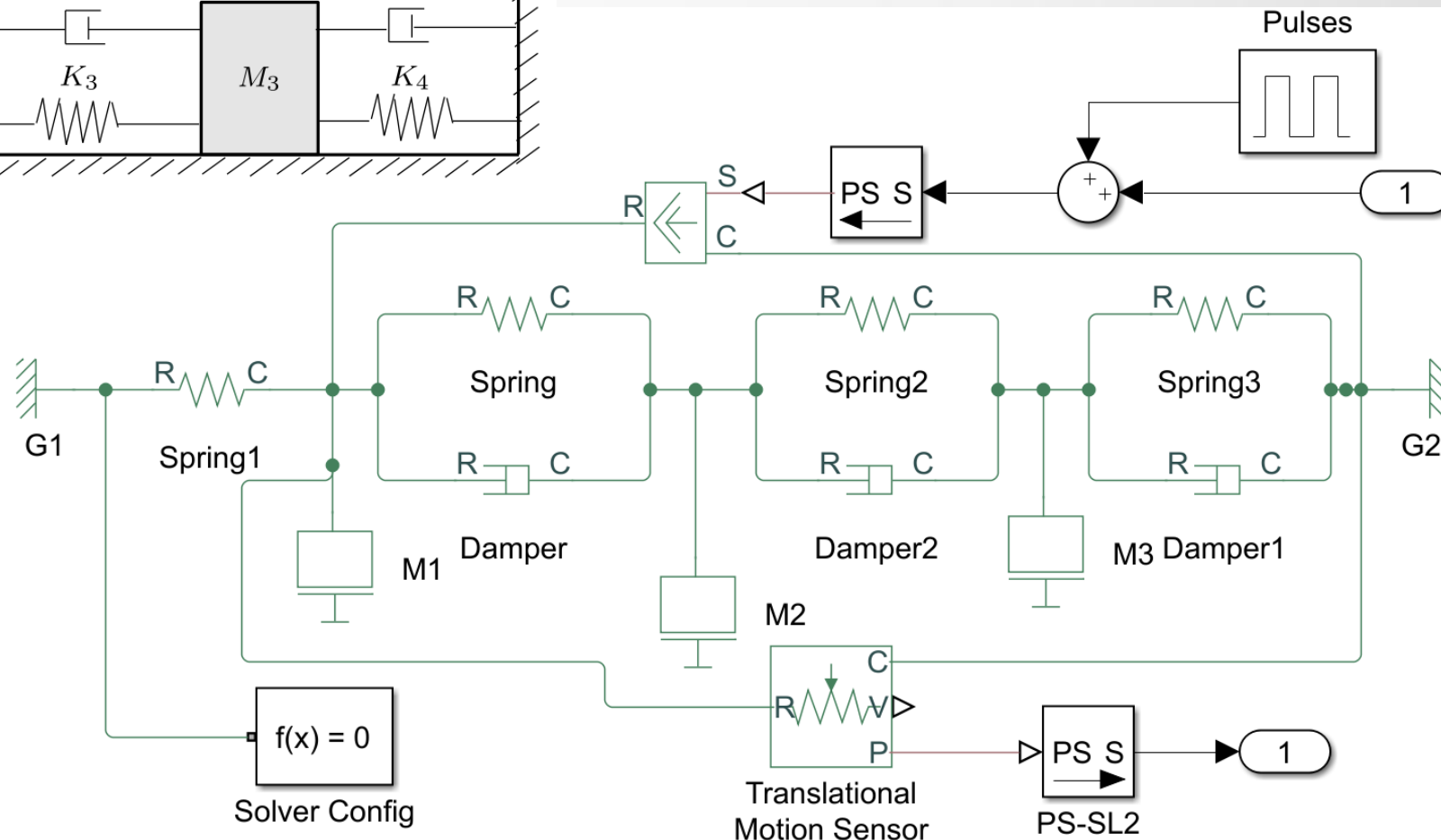
➤ 重物、弹簧、阻尼器、支点、输入与输出



Simulink物理建模



- 物理模型建立
- 元件直接连接
- 外力添加
- 位移检测
- 信号的转换





系统的仿真分析

➤ 系统的仿真模型

- 事先建立的模型：c6mdamp.mdl
- 在方波外力作用下模块 M_1 的位移

➤ 绘制位移曲线

 `>> [t,x,y]=sim('c6mdamp'); plot(t,y)`

➤ 线性化

 `>> G=linearize('c6mdamp'); G=minreal(G)
G1=zpk(G)`



多领域物理建模小结

- 底层建模很麻烦
 - 需要先写出数学模型
 - 写数学模型需要领域的知识
- 多领域物理建模的思想
 - 像搭建硬件系统那样将组件组装起来
 - 自动写出仿真模型，直接仿真
- 进一步阅读
 - 薛定宇、陈阳泉《基于MATLAB/Simulink的系统仿真技术与应用》，清华大学出版社，2011

